



PDF Download
3704998.pdf
06 April 2026
Total Citations: 12
Total Downloads:
1973

Latest updates: <https://dl.acm.org/doi/10.1145/3704998>

RESEARCH-ARTICLE

Market-aware Long-term Job Skill Recommendation with Explainable Deep Reinforcement Learning

YING SUN, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

YANG JI, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

HENGSHU ZHU, Chinese Academy of Sciences, Beijing, Beijing, China

FUZHEN ZHUANG, Beihang University, Beijing, China

QING HE, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

HUI XIONG, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

Open Access Support provided by:

Institute of Computing Technology Chinese Academy of Sciences

Chinese Academy of Sciences

Hong Kong University of Science and Technology

Beihang University

Published: 22 January 2025
Online AM: 21 November 2024
Accepted: 12 November 2024
Revised: 22 September 2024
Received: 05 January 2024

Citation in BibTeX format

Market-aware Long-term Job Skill Recommendation with Explainable Deep Reinforcement Learning

YING SUN and YANG JI, Thrust of Artificial Intelligence, The Hong Kong University of Science and Technology—Guangzhou Campus, Guangzhou, China

HENGSHU ZHU, Computer Network Information Center, Chinese Academy of Sciences, Beijing, China

FUZHEN ZHUANG, Beihang University, Beijing, China

QING HE, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China

HUI XIONG, The Hong Kong University of Science and Technology, Hong Kong, Hong Kong and Thrust of Artificial Intelligence, The Hong Kong University of Science and Technology—Guangzhou Campus, Guangzhou, China

Continuously learning new skills is essential for talents to gain a competitive advantage in the labor market. Despite extensive efforts on relevance- or preference-based skill recommendations, little attention has been given to the practical effects of job skills in the market. To bridge this gap, we propose an explainable personalized skill learning recommendation system that considers the long-term learning benefits and costs. Specifically, we model skill learning utilities based on salary and learning cost associated with job positions and propose a multi-objective deep reinforcement learning framework to model and maximize long-term utilities. Furthermore, we propose a Self-explaining Skill Recommendation Deep Q-network (SeSRDQN) that captures and prototypes prevalent skill sets in the market into representative exemplars for decision-making. SeSRDQN quantitatively decomposes the talent's long-term learning utility into contributions from each exemplar, offering a comprehensive and multi-factorial explanation across various skill learning options. To tackle the combinatorial complexity of the skill space, we develop an MCTS-based optimization-decoding iterative training procedure for explanation fidelity and human understandability. In this way, talents will receive a tailored roadmap of essential skills, complemented by exemplar-based explanations, to effectively plan their careers. Extensive experiments on a real-world dataset validate the effectiveness and explainability of our approach.

Ying Sun and Yang Ji are co-first authors and contribute equally to this work.

This is a substantially extended and revised version of [60], which appears in the *Proceedings of the Web Conference 2021*. This work is partly supported by the National Key Research and Development Program of China (No. 2023YFF0725001), the National Natural Science Foundation of China (Nos. 62306255, 92370204, 62176014), the Natural Science Foundation of Guangdong Province (No. 2024A1515011839), the Fundamental Research Project of Guangzhou (No. 2024A04J4233), the Guangzhou-HKUST(GZ) Joint Funding Program (No. 2023A03J0008), and the Education Bureau of Guangzhou Municipality. Authors' Contact Information: Ying Sun, Thrust of Artificial Intelligence, The Hong Kong University of Science and Technology—Guangzhou Campus, Guangzhou, China; e-mail: yings@hkust-gz.edu.cn; Yang Ji, Thrust of Artificial Intelligence, The Hong Kong University of Science and Technology—Guangzhou Campus, Guangzhou, China; e-mail: yji655@connect.hkust-gz.edu.cn; Hengshu Zhu (corresponding author), Computer Network Information Center, Chinese Academy of Sciences, Beijing, China; e-mail: zhuhengshu@gmail.com; Fuzhen Zhuang, Beihang University, Beijing, China; e-mail: zhuangfuzhen@buaa.edu.cn; Qing He, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China; e-mail: heqing@ict.ac.cn; Hui Xiong (corresponding author), The Hong Kong University of Science and Technology, Hong Kong, Hong Kong and Thrust of Artificial Intelligence, The Hong Kong University of Science and Technology—Guangzhou Campus, Guangzhou, China; e-mail: xionghui@ust.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1558-2868/2025/1-ART46

<https://doi.org/10.1145/3704998>

CCS Concepts: • **Information systems** → **Data mining**;

Additional Key Words and Phrases: Skill Recommendation, Explainable Recommendation, Reinforcement Learning, Deep Q Network, Case-based Reasoning, Prototype Learning, Self-Interpretable Neural Network

ACM Reference format:

Ying Sun, Yang Ji, Hengshu Zhu, Fuzhen Zhuang, Qing He, and Hui Xiong. 2025. Market-aware Long-term Job Skill Recommendation with Explainable Deep Reinforcement Learning. *ACM Trans. Inf. Syst.* 43, 2, Article 46 (January 2025), 35 pages.
<https://doi.org/10.1145/3704998>

1 Introduction

In the world of Volatility, Uncertainty, Complexity, and Ambiguity, talents are required to possess more specialized job skills to meet the reality of the increasingly refined division of labor and higher levels of work intensification. As shown in recent studies, possessing advanced job skills can help talents significantly increase their salary in the labor market [10, 14, 61]. Therefore, learning new skills becomes essential for talents to keep up with the fast-moving technical trends, which diversifies their job options and builds competitive advantage. However, due to the dynamic nature and the uncertain benefits of skill learning, there is still a lack of sustainable guidance for talents to learn the right and appropriate job skills.

During the past decades, large efforts have been devoted to skill recommendation for talents [7, 13, 39, 44, 46, 65]. Nevertheless, these works mainly focus on measuring the relevance of job skills to the current occupations of talents for making recommendations, while the multi-faceted skill learning utilities, such as the learning cost and the potential benefit, have been largely neglected. Meanwhile, existing skill recommendation approaches often operate in a static manner, failing to account for the long-term impact of skill acquisition on career development. Sociological findings suggest that long-term planning in skill acquisition leads to better career outcomes and higher job satisfaction [9, 63]. From the perspective of sustainable talent development, the skills selected should provide long-term benefits and maintain an acceptable learning cost. For instance, as illustrated in Figure 1, for a talent with basic **Machine Learning (ML)** skills, acquiring a fundamental skill, such as Statistics, requires more time and effort initially, but it offers continuous and long-term opportunities for skill enhancement and sustained career advancement. This contrasts with learning a simpler skill like Web design, which might provide immediate benefits but has limited long-term growth potential.

For a skill recommendation system, it is crucial to explain the recommendation results in terms of explicit multi-faceted effects of skill learning. This not only helps users build and calibrate trust in the system but also assists them in making informed and rational career decisions. However, quantifying these factors and making a comprehensive decision is a non-trivial task. First, since skills have complex interactions, the benefits can vary significantly for talents with distinct skill sets. For example, adding data analytics skills might greatly enhance a junior ML developer's career prospects but offer limited benefits to a hardware developer. Second, since there are an exponentially large number of possible skill combinations on the learning path, the long-term benefit of skill learning is abstract and hard to model. While skills required for high-paying jobs may offer substantial long-term benefits, the learning path to reach such jobs can be prohibitively difficult. Third, the complex patterns of skills' multi-faceted future effects can be subtle and unclear, which complicates users' ability to understand and trust the recommendation results when making actual decisions. Therefore, explicitly representing these intricate influence patterns and providing concrete evidence for long-term benefit estimations is both significant and challenging.

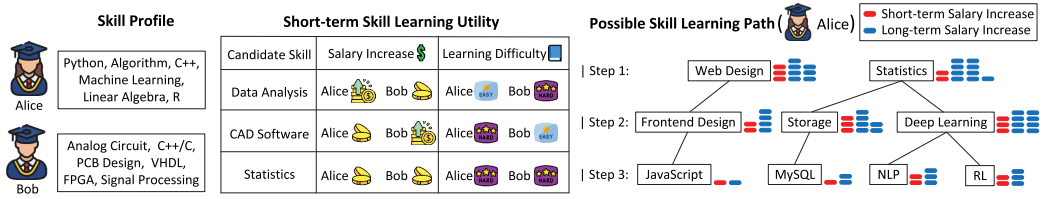


Fig. 1. A motivated example of long-term skill recommendation.

To tackle these challenges, we propose a data-driven solution to provide long-term, cost-effective, and explainable recommendations for talents who want to acquire new job skills. Specifically, to quantify skill learning utilities under various circumstances, we build a skill learning environment, including a skill-matching-based salary estimator and a frequent itemset-based learning difficulty estimator. On this basis, we propose a multi-objective deep **Reinforcement Learning (RL)**-based recommendation framework, namely **Skill Recommendation Deep Q-network (SRDQN)** which provides multi-aspect skill selection evidence by estimating the long-term utilities of different learning decisions. To further raise the model's explainability of the intrinsic reasoning process for long-term utilities, we propose a **Self-explaining SRDQN (SeSRDQN)**, which draws future learning utilities for the talent skill sets by contrasting them with a selection of representative examples. Specifically, considering the multi-faceted talent skill sets and the progressive nature of DQN training algorithms, we propose a novel frequent subset-guided prototyping method to capture the prevalent skill sets in the market that can perform as representative exemplars for decision-making. Moreover, we design an MCTS-based optimization-decoding iterative training method to effectively learn and explore the learned prototypical skill sets under the discrete and combinatorial nature of the large skill set space. With these well-aligned designs, SeSRDQN is crafted to provide intrinsic explainability for the recommendation results while preserving competitive performance relative to black-box models. Finally, we conduct extensive experiments on a real-world recruitment dataset. The experimental results clearly demonstrate that our method outperforms existing methods in generating meaningful and beneficial skill learning paths. Furthermore, concrete case studies, visualizations, and user studies demonstrate that SeSRDQN provides insights into the decision-making rationality relevant to different learning choices.

Compared with the preliminary work [60], we focus on building a self-explaining and well-performing deep RL model to reveal the inner decision-making process of skill recommendation. Firstly, we have formulated our original model as a generic structure for long-term skill learning utility estimation. Secondly, we have proposed a novel SeSRDQN, which provides an exemplar-based explanation for recommendation results. In particular, we have provided detailed designs on the network structure, training methodology, and prototype projection. Accordingly, we have conducted substantial additional experiments on the effectiveness and explainability of SeSRDQN and provided insights into the model's behavior through thorough performance evaluation, case studies, and user studies.

2 Preliminaries

This section introduces the preliminaries, including problem formulation and framework overview. The major notations of this article are presented in Table 1.

2.1 Problem Formulation

We utilize online job advertisement data [61] to realize our skill recommendation system. Each job advertisement consists of the offered job salary and the textual descriptions of skill requirements.

Table 1. Major Notations in This Article

Notation	Explanation
O^t	The skill set (state) at the t th step
o_i	The i th element in skill set O
s^t	The selected skill (action) at the t th step
J^j	The j th job
R^j	The skill requirement of the j th job
r_i^j	The i th skill in R^j
m^j	The offered salary of the j th job
$f_{sal}(O)$	The salary estimation of skill set O
$f_p(O, J^j)$	The probability of talent getting job J^j with O
$f_e(O, J^j)$	The expected salary that O can get from J^j
$f_{dif}(s, O)$	The difficulty of learning skill s based on O
w	Parameter of the sigmoid function
θ_{sal}	The skill coverage threshold for getting a job in the salary estimator
$p(s O)$	The appearing probability of skill s given O
$p(s)$	The probability of skill s appearing in job postings
θ_{dif}	The probability threshold for being a frequent itemset
$\mathcal{F}^t$	The frequent subsets at the t th step
F_i^t	The i th element in $\mathcal{F}^t$
$r(s O)$	The reward of learning s under state O
$r_{sal}(s O)$	The salary increase of learning s under state O
$r_{dif}(s O)$	The difficulty of learning s under state O
π	Policy
$Q_\pi(s O)$	The Q-value of action s at state O under policy π
$Q_\pi^{sal}(s O)$	The salary Q-value of taking action s at state O under policy π
$Q_\pi^{dif}(s O)$	The difficulty Q-value of taking action s at state O under policy π
$Q_u(s, p_i)$	The long-term prototype utility for the prototype p_i with respect to the skill s
$Sim(O, p_i)$	The similarity score between a prototype p_i and an input skill set O
γ	Discount factor
α	Difficulty importance factor
λ_{freq}	Parameter of the clustering and evidence regularization terms
λ_{div}	Parameter of the diversity regularization term
λ_{enc}	Parameter of the decoding regularization term
$\mathcal{N}$	The Monte Carlo tree node
N_c	The candidate pool size
$C(O)$	The candidate pool of state O
$\mathcal{F}_O$	The frequent itemsets under the state O
fr	Frequent itemset
X_O	The embedding of the input state O
X_{fr}	The embedding of the frequent itemset fr
N_s	The total number of skills
N_J	The total number of job postings
k	The prototype number

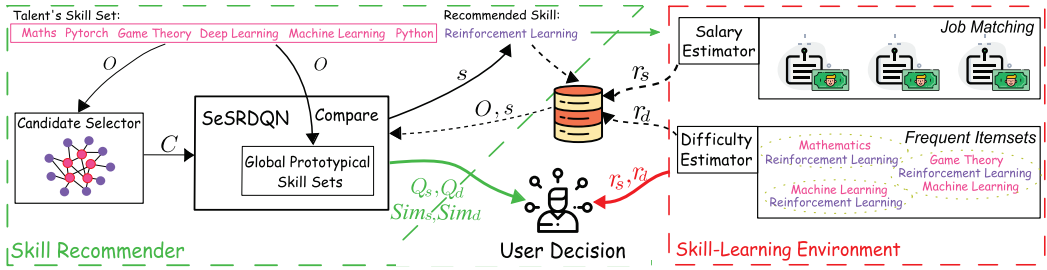


Fig. 2. The overview of deep RL-based sustainable skill recommendation.

We extract the job skills into a set and formulate the job advertisement as $J^j = (R^j, m^j)$, where m^j denotes the job salary and $R^j = \{r_1^j, r_2^j, \dots, r_k^j\}$ denotes the skill set. Given a talent skill set $O = \{o_1, o_2, \dots\}$, our problem includes three parts: (1) how to quantify the profit and cost of skill learning with online job advertisement data; (2) how to recommend a sequence of skills that continuously bring high profit to the talent with low cost; and (3) how to unravel and explain the evidence of estimating the talent skill sets' effect on the future utilities relevant to each choice.

2.2 Framework Overview

Figure 2 shows the overview of our skill recommendation framework. The first part of this work is a *skill-learning environment*. It simulates the salary increase (profit) and learning difficulty (cost) by mining skill interactions in job advertisements. Motivated from person-job fit theory [8, 55] that fitness between talent ability and job requirement decides the job competency, the salary estimator (Section 3.1) estimates salary according to the offered salary of jobs that matches the talent skill set. Motivated from learning transfer theory [6, 18] that owning related knowledge make learning a new skill easier, the difficulty estimator (Section 3.2) measures learning difficulty in terms of the skill's relevance with the current skill set.

Our main purpose is to recommend a sequence of skills that bring high profit to the talent with low cost in the long term. To achieve this, we formulate this skill recommendation task as a *multi-objective RL problem* (Section 4.1), where salary increase and learning difficulty are treated as separate reward signals. We then introduce a *generic skill recommendation framework based on deep RL*, named SRDQN (Section 4.2), which learns a comprehensive policy optimized for long-term returns. SRDQN has a multi-head structure that separately estimates the long-term utilities of salary increase and learning difficulty. For each part, we can design flexible and adaptable Q-value estimation strategies to support disentangled analysis for explaining the personalized recommendation results. To enhance training efficiency, a *candidate selector* filters the candidate skills that are densely connected with the talent's skills on a skill graph.

To enhance the inherent explainability of the recommender, we further propose a SeSRDQN (Section 4.3) under the SRDQN framework. Specifically, SeSRDQN introduces *prototypical skill set-based Q-value estimation*, enabling a transparent decision-making process. The Q-value of each candidate skill is computed by comparing the talent's skill set with a set of representative prototypes and combining the similarity scores. Then, the *frequent subset-guided prototyping approach* coupled with a *MCTS-based optimization-decoding iterative training procedure* is proposed to effectively align these prototypes with human-understandable and meaningful skill sets in the market. In this way, SeSRDQN reveals how talents' current skill subsets influence the long-term learning utilities under different choices. By comparing the global prototypical skill sets and candidate skill learning

choices, talents can align their understanding with the model's internal mechanisms, enabling them to craft more effective career plans.

3 Skill Learning Environment

This section introduces salary and difficulty utility models based on job advertisement data and their fast implementations to raise training efficiency.

3.1 Salary Estimator

We match the talent skill set O with the job requirement set R^j to find out jobs that the talent is qualified for and estimate the talent salary $f_{sal}(O)$ according to their offered salary. Specifically, we measure $f_{sal}(O)$ as the averaged top-K salary of the matched jobs. Intuitively, O is *qualified* for J^j if $R^j \subseteq O$. However, such hard qualification can bring sharp changes in salary estimation when learning a new skill, causing difficulty in model convergence. Therefore, we define *soft qualification* with the intuition that a talent owning most required skills has the possibility to be qualified for the job. We model the probability as $f_p(O, J^j) = (1 + \exp(-w(\frac{|R^j \cap O|}{|R^j|} - \frac{1}{2})))^{-1}$, where $w \in \mathbb{R}$ is a hyper-parameter. A large w ensures $f_p(O, J^j)$ to be close to 1 when $R^j \subseteq O$ and close to 0 when $R^j \cap O = \emptyset$. Motivated by real recruiting scenarios, we assume the talent cannot get the job when $\frac{|R^j \cap O|}{|R^j|}$ is less than a threshold θ_{sal} . Notably, it degenerates to the hard qualification when θ_{sal} is set to 1. In this way, the expected salary that O can get from J^j is $f_e(O, J^j) = \mathbb{1}\{\frac{|R^j \cap O|}{|R^j|} \geq \theta_{sal}\} m^j f_p(O, J^j)$. Then, we estimate $f_{sal}(O)$ as the averaged top-K expected salary, which is a monotonic function. Specifically, for any O and $o' \notin O$, we have $\forall j, f_e(O, J^j) \leq f_e(O \cup \{o'\}, J^j)$. It means that, for each of the top-K jobs, the expected salary will not decrease. Thus we have $f_{sal}(O) \leq f_{sal}(O \cup \{o'\})$. The monotonicity agrees with common sense that owning more skills leads to higher-paid jobs.

For each session of the sequential recommendation, the qualified jobs for each step need to be retrieved from the entire job set to measure the corresponding salary. Going through the job set in every step brings high computational complexity of $O(ML)$, where $M = \sum_{j=1}^{N_j} |R^j|$ and L is the number of recommended skills. To accelerate this process, we incrementally update the skill coverage status of the jobs. Specifically, for each skill i , we store the jobs requiring it with set S_i . When learning a new skill i , we only update the coverage status of jobs in S_i . The space requirement of this process is $O(M)$, which is the same as the job storage and raises no space complexity. Finally, we maintain a max-heap to store the expected salary of each job and get the top-K in $O(K \log(N_j))$. The detailed algorithm is shown in Algorithm 1.

3.2 Learning Difficulty Estimator

The relevant skills in the current skill set influence the difficulty of learning a new skill. For example, talents familiar with “Java” can more easily learn “Scala”. Therefore, we propose a relevance-based difficulty measurement, formulated as $f_{dif}(s, O) = -\max_{O_s \subseteq O} \log p(s|O_s)$, where $p(s|O_s)$ denotes the probability of a skill s appearing in a job requiring a skill set O_s . The probability can be estimated

with Maximum Likelihood Estimation as $p(s|O_s) = \frac{\sum_{j=1}^{N_j} \mathbb{1}\{\{s\} \cup O_s \subseteq R^j\}}{\sum_{j=1}^{N_j} \mathbb{1}\{O_s \subseteq R^j\}}$, where $p(s) = \frac{\sum_{j=1}^{N_j} \mathbb{1}\{s \in R^j\}}{N_j}$.

Notably, $p(s|\emptyset) = p(s)$. Corresponding to the fact that mastering more skills decreases the difficulty of learning a new skill, f_{dif} is monotonic. Specifically, given any skill $a \notin O \cup \{s\}$, we have the

Algorithm 1: Sequential Job Matching Algorithm

Input: $R^1, \dots, R^{N_j}$: jobs formed as skill sets; $m^1, \dots, m^{N_j}$: job salary; $o_1, \dots, o_L$: input skill sequence; $topK$: number of top salary to be averaged; N_s : number of skills; θ_{sal} : probability threshold; w : probability parameter;

Output: $salary_1, \dots, salary_L$: salary prediction for each step;

- 1: Initialize N_s empty lists $S_1, S_2, \dots, S_{N_s}$;
- 2: **for** $j \leftarrow 1 \dots N_j$ **do**
- 3: $count_j \leftarrow 0$;
- 4: **for each** $s \in R^j$ **do**
- 5: $S_s \leftarrow S_s \cup \{j\}$;
- 6: $heap \leftarrow$ empty max-heap ;
- 7: $O' \leftarrow$ empty set;
- 8: **for** $i \leftarrow 1 \dots L$ **do**
- 9: **for each** $j \in S_{o_i}$ **do**
- 10: $count_j \leftarrow count_j + 1$;
- 11: **if** $\frac{count_j}{|R^j|} \geq \theta_{sal}$ **then**
- 12: Add $(job=j, val = \frac{m^j}{1+\exp(-w(\frac{count_j}{|R^j|} - \frac{1}{2}))})$ to $heap$;
- 13: $salary_i \leftarrow 0$;
- 14: $storeList \leftarrow$ empty list;
- 15: **for** $k \leftarrow 1 \dots topK$ **do**
- 16: $x \leftarrow$ pop $heap$;
- 17: **if** $x.job \notin storeList$ **then**
- 18: $storeList \leftarrow storeList \cup \{x.job\}$;
- 19: $salary_i \leftarrow salary_i + x.val/K$;
- 20: Clear $heap$;
- 21: **for each** $j \in storeList$ **do**
- 22: Add $(job = j, val = \frac{m^j}{1+\exp(-w(\frac{count_j}{|R^j|} - \frac{1}{2}))})$ to $heap$;

return $salary_1, \dots, salary_L$;

relevance-based difficulty estimation:

$$\begin{aligned}
 f_{dif}(s, O \cup \{a\}) &= - \max_{O_s \subseteq O \cup \{a\}} \log p(s|O_s) \\
 &= - \max_{O_s \in O} \max\{\log p(s|O_s), \log p(s|O_s \cup \{a\})\} \leq f_{dif}(s, O).
 \end{aligned} \tag{1}$$

Going through all the possible subsets of O to calculate $f_{dif}(s, O)$ is infeasible since there are $2^{|O|} - 1$ subsets in total. To reduce time complexity, we use frequent itemsets to approximate the difficulty. Specifically, since $p(s|O_s) = \frac{p(\{s\} \cup O_s)}{p(O_s)}$ is meaningless if O_s seldom appears in jobs, we eliminate the infrequent subsets and approximate the difficulty as $f_{dif}(s, O) = - \max_{O_s \subseteq O, p(O_s) \geq \theta_{dif}} \log p(s|O_s)$, where θ_{dif} is a pre-defined threshold. For the sequential recommendation, frequent subsets in each step need to be maintained for measuring the corresponding learning difficulty. With the property that a frequent itemset's subset is also frequent, we incrementally update frequent subsets in each step as $\mathcal{F}^t = \{F_0^t, F_1^t, \dots\}$. Specifically, we add the new skill to each element in $\mathcal{F}^{t-1}$ and check if the new set is frequent, formed as $\mathcal{F}^t = \mathcal{F}^{t-1} \cup \{F \cup \{s\} | F \in \mathcal{F}^{t-1}, p(F \cup \{s\}) > \theta_{dif}\}$.

Although we have largely reduced the number of subsets, going through $\mathcal{F}^t$ still has high time complexity. To solve this problem, we propose a *graph-based difficulty estimation algorithm*, which can calculate the difficulty and find out $\mathcal{F}^{t+1}$ without going through all the elements in $\mathcal{F}^t$.

Algorithm 2: Difficulty Estimation Algorithm

Input: $F_0, F_1, \dots, F_{N_f}$: a list of frequent skill sets, F_0 is the empty set; $prob_1, \dots, prob_{N_f}$: the probability of the skill sets; $o_1, o_2, \dots, o_L$: input skill sequence; θ_{dif} : probability threshold; N_s : number of skills;

Output: $d_1, \dots, d_L$: difficulty estimation of each step;

```

1: Initialize empty sets  $Edges_0, \dots, Edges_{N_f}$ ;
2: for  $j = 1 \rightarrow N_f$  do
3:   for each  $s \in F_j$  do
4:      $i \leftarrow$  index of frequent set  $F_j - \{s\}$ ;
5:     Add edge  $\langle start=i, end=j, skill=s \rangle$  to  $Edges_i$ ;
6: Initialize empty sets  $Active_1, \dots, Active_{N_s}$ ;
7: for each edge  $\in Edges_0$  do
8:   Add edge to  $Active_{edge.skill}$ ;
9: for each  $i = 1 \rightarrow L$  do
10:   $d_i = +\infty$ ;
11:  for each edge  $\in Active_{o_i}$  do
12:     $d_i \leftarrow \min(-\log(\frac{prob_{edge.end}}{prob_{edge.start}}), d_i)$ ;
13:    for each nextEdge  $\in Edges_{edge.end}$  do
14:      Add nextEdge to  $Active_{edge.skill}$ ;
return  $d_1, \dots, d_L$ ;

```

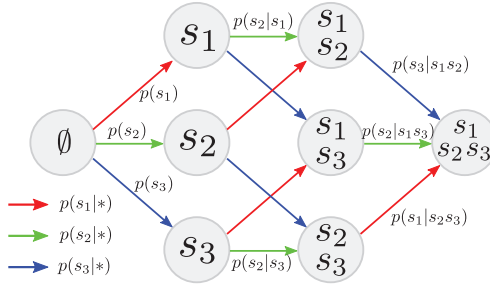


Fig. 3. Probabilistic graph for frequent skill sets.

Specifically, we find all the possible frequent skill sets (including the empty set) in advance with *frequent itemset mining algorithm* [26]. Then, we organize the skill sets with a graph structure [27, 33], formed like the example shown in Figure 3. In this graph, each node represents a frequent skill set, and a directed edge $\langle start, end \rangle$ exists between node $start$ and node end if $|F_{end} - F_{start}| = 1$. Supposing s is the *incremented skill* on the edge (i.e., $\{s\} \cup F_{start} = F_{end}$), we can calculate a conditional probability with this edge as $p(s|F_{start}) = \frac{p(F_{end})}{p(F_{start})}$. To sequentially estimate the learning difficulty, we start from an empty set and update the graph with the skills one-by-one. Specifically, we regard nodes in $\mathcal{F}^t$ as activated and group their out-edges by incremented skills. When adding a skill, only the out-edges in the corresponding group contribute to its learning difficulty. Meanwhile, the end nodes of these edges are the newly found frequent subsets. We activate these nodes to obtain $\mathcal{F}^{t+1}$. We describe the details of this process in Algorithm 2. Notably, all the edges are activated at most once, so the time cost is approximately the number of the total activated edges, which is $O(\sum_{F \in \mathcal{F}^*} |F|)$, where $\mathcal{F}^*$ denotes the frequent subsets of the complete skill set.

4 Methodology

In this section, we first formulate the job skill recommendation task as a multi-objective RL problem. We then introduce the generic framework of our proposed SRDQN, designed to learn an effective

skill learning strategy. Building on SRDQN, we further develop the SeSRDQN, which enhances transparency by providing users with intrinsic insights into the decision-making process.

4.1 Multi-objective RL Formulation

We formulate the skill recommendation as a sequential decision-making process with two skill-learning objectives. At the t th step of skill selection, we define *state* as the talent's current skill set O_t and *action* as learning a new skill s_t . The *state dynamic* can be written as $O_{t+1} = \text{trans}(s_t, O_t) = O_t \cup \{s_t\}$. Taking an action brings two rewards, including the salary increase $r_{sal}(s_t|O_t) = f_{sal}(O_t \cup \{s_t\}) - f_{sal}(O_t)$ and the learning difficulty $r_{dif}(s_t|O_t) = f_{dif}(s_t, O_t)$. At each step, supposing the talent continues learning with a probability γ ($0 < \gamma < 1$) or stops otherwise, the expected total future salary increase and difficulty for state O_t can be written as

$$\begin{aligned} G_{sal}^t &= r_{sal}(s_t|O_t) + \sum_{k=1}^c \gamma^k r_{sal}(s_{t+k}|O_{t+k}), \\ G_{dif}^t &= r_{dif}(s_t|O_t) + \sum_{k=1}^c \gamma^k r_{dif}(s_{t+k}|O_{t+k}), \end{aligned} \quad (2)$$

where c is the horizon. Under RL formulation, by regarding γ as the discount factor, G_{sal}^t and G_{dif}^t are, respectively, the long-term salary and difficulty return. Moreover, we use Q-value $Q_\pi(s_t|O_t)$ to represent the specific long-term return when taking action s_t under state O_t following policy π . The Bellman Equation [43] of the salary Q-value $Q_\pi^{sal}(s_t|O_t)$ and difficulty Q-value $Q_\pi^{dif}(s_t|O_t)$ can be separately formulated as

$$\begin{aligned} Q_\pi^{sal}(s_t|O_t) &= r_{sal}(s_t|O_t) + \gamma Q_\pi^{sal}(s_{t+1}|O_{t+1}), \\ Q_\pi^{dif}(s_t|O_t) &= r_{dif}(s_t|O_t) + \gamma Q_\pi^{dif}(s_{t+1}|O_{t+1}). \end{aligned} \quad (3)$$

To balance the two objectives, i.e., maximizing G_{sal}^t and minimizing G_{dif}^t , with a unified policy, we define the total long-term return as $G^t = G_{sal}^t - \alpha G_{dif}^t$, where $\alpha \in \mathbb{R}$ denotes a pre-defined difficulty importance factor. The Bellman Equation can be written as $Q_\pi(s_t|O_t) = r(s_t|O_t) + \gamma \max_{s'} Q_\pi(s'|O_{t+1})$, which is equal to combining the Q-value function of different objectives, i.e., $Q_\pi(s_t|O_t) = Q_\pi^{sal}(s_t|O_t) - \alpha Q_\pi^{dif}(s_t|O_t)$. As a common practice, we set a deterministic policy of choosing the action with the highest Q-value, i.e., $s_{t+1} = \pi(O_t) = \arg \max_s Q_\pi(s|O_t)$.

4.2 SRDQN

In this part, we describe SRDQN in detail, whose structure is shown in Figure 4. Considering the distinct utilities of salary increase and learning difficulty, SRDQN estimates Q^{sal} and Q^{dif} separately with a specially designed multi-task structure. The encoder layer learns representations for each candidate skill (action) and the overall talent skill set (state) as X_s and $X_O = \sum_{o \in O} X_o$, respectively. Then, each utility part combines X_s with X_O to model a Q-value. In the last layer, SRDQN combines the two Q-values to obtain a unified recommendation policy. This multi-task structure not only takes advantage of better model performance but also provides the users with separate long-term utility information, which increases model explainability. It is notable that SRDQN is a generic structure for multi-objective long-term learning utility estimation. It offers the flexibility to incorporate various neural network structures for Q-value estimation, tailored to different domain- or user-specific needs. In particular, to further enhance user understanding of the internal Q-value estimation process, we propose SeSRDQN, which will be detailed in the next part.

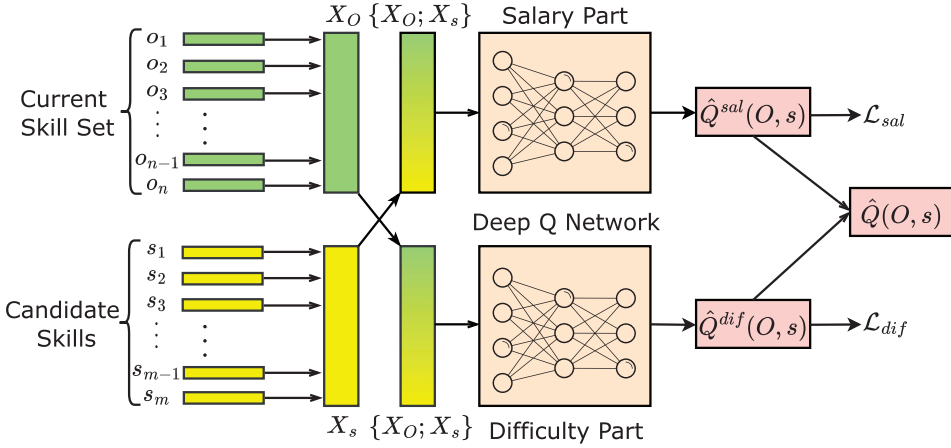


Fig. 4. Framework of SRDQN.

The model is trained iteratively with the bootstrap method [43], where the training labels are constructed based on the estimation of the previous model. Specifically, the loss functions of the Bellman error for the salary estimator and difficulty estimator can be formulated as

$$\begin{aligned} \mathcal{L}_{sal} &= \mathbb{E}_{O, s \sim \mu(\cdot, O; \Phi^-)} [(r_{sal}(s|O) + \gamma \max_{s' \in C(O')} \hat{Q}^{sal}(s', O'; \Phi^-) - \hat{Q}^{sal}(s, O; \Phi))^2], \\ \mathcal{L}_{dif} &= \mathbb{E}_{O, s \sim \mu(\cdot, O; \Phi^-)} [(r_{dif}(s|O) + \gamma \max_{s' \in C(O')} \hat{Q}^{dif}(s', O'; \Phi^-) - \hat{Q}^{dif}(s, O; \Phi))^2], \end{aligned} \quad (4)$$

where $O' = O \cup \{s\}$, Φ and Φ^- are the current and previous model parameters, $\hat{Q}^{sal}$ and $\hat{Q}^{dif}$ are the estimated salary and difficulty Q-value functions, $\mu(\cdot, O'; \Phi^-)$ stores the chosen probability of each skill given state O' under the current policy. We combine these two parts into the model's loss function as $\mathcal{L}_{rew} = \mathcal{L}_{sal} + \alpha \mathcal{L}_{dif}$, where $\alpha \in \mathbb{R}$ is a super-parameter balancing the importance of these two losses. During training state, we use the ϵ -greedy strategy [69] to sample actions from the candidate skills. Formally, the choice probability $\mu(s, O)$ of skill s given state O can be formulated as

$$\mu(s, O; \Phi^-) = \begin{cases} \frac{1-\epsilon}{|C(O)|} & s \neq \arg \max_{s' \in C(O)} \hat{Q}(s', O; \Phi^-) \\ \epsilon + \frac{1-\epsilon}{|C(O)|} & s = \arg \max_{s' \in C(O)} \hat{Q}(s', O; \Phi^-), \\ 0 & s \notin C(O) \end{cases} \quad (5)$$

where $C(O)$ is the candidate skill pool selected from the skill set O .

Candidate Selection. The skill recommendation problem has a large action space. Considering all the skills for decision-making brings up not only high time complexity but also model convergence difficulty. Intuitively, skills less relevant to the current skill set are less likely to be good options because they have higher learning difficulties and rarely bring significant salary increases within limited learning steps for not being needed by the industry where the talent is currently in. Therefore, we propose a graph-based candidate skill selector to filter skills related to the current skill set. Specifically, we construct a skill graph where each node represents a skill. An edge connects two skills if their co-appearance frequency is large enough. We count each skill's neighboring nodes in the current skill set as a rough measurement of the relevance and pick N_c skills with the highest relevance as the candidates. The model only selects skills from the candidate pool during decision-making, which not only reduces the time complexity for Q-value estimation but also raises training efficiency.

Dual Layers for Deterministic State Dynamics. Notably, the *state dynamics* of our proposed skill recommendation problem is deterministic, i.e., the next state is determined when state and action are determined. Moreover, it is common that two different states transit to the same state. For example, both state $O_0 = \{s_1, s_2\}$ and $O_1 = \{s_1, s_3\}$ transit to $O' = \{s_1, s_2, s_3\}$ by learning skill s_3 and s_2 , respectively. In this case, supposing s^* is the best action under state O' , we have $Q(s_3|O_0) = r(s_3|O_0) + \gamma Q(s^*|O')$ and $Q(s_2|O_1) = r(s_2|O_1) + \gamma Q(s^*|O')$. For ease of presentation, here we omit the subscript distinguishing salary and difficulty. Notably, the two equations share the second term, which is a function of the next state. Therefore, we model $Q(s|O)$ as $h(s, O; \Phi_h) + g(O \cup \{s\}; \Phi_g)$, where Φ_h and Φ_g are the model parameters. In this way, SRDQN can raise model performance by seizing the relation between Q-value of different states. Specifically, we concatenate $X(O)$ and $x(s)$ and use **Multi-layer Perceptron (MLP)** to model $h(s, O; \Phi_h)$. Observing that $X(O \cup \{s\}) = \sum_{o \in O \cup \{s\}} x(o) = X(O) + x(s)$, we sum up $X(O)$ and $x(s)$ to obtain the embedding of the next state. Then, we use MLP to model $g(O \cup \{s\}; \Phi_g)$. Specifically, since difficulty is measured with conditional probability, while salary is measured by directly matching the job postings. This causes the measured difficulty smoother than salary. The smoothness largely reduces the long-term uncertainty of difficulty and makes it more easily modeled. Therefore, we only apply this structure to model Q^{sal} and simply use the concatenated vector to model Q^{dif} .

4.3 SeSRDQN

Explainable recommendation is crucial for personalized skill recommenders by answering why-type questions from talents, which improves the system transparency and user satisfaction. However, traditional DQNs cannot well support the quantitative utility modeling, and thus limits their ability to support transparent reasoning. To bridge this gap, we develop SeSRDQN, which learns representative skill compositions as prototypes, each associated with specific job market semantics. By comparing these prototypes with the input skill set and integrating their similarities into the Q-value estimation, SeSRDQN could provide inherently explainable recommendations grounded in clear and understandable logic.

The architecture of our SeSRDQN is shown in Figure 5. Following the multi-objective rationale of SRDQN, SeSRDQN estimates salary Q-value Q^{sal} and difficulty Q-value Q^{dif} separately and combines them into a unified policy. For ease of presentation, we omit the description distinguishing salary and difficulty. The key design components of SeSRDQN are introduced as follows:

(1) *Prototypical Skill Set-based Q-value Estimation.* A talent's skill set can be viewed as a combination of skill clusters that represent typical abilities for multiple different areas. For instance, talents in data science roles, such as Web analysts, often exhibit a multi-faceted skill set encompassing both data analysis and Web development. When they choose skills to learn, skills of different perspectives may have relatively decoupled influences on future learning utilities. Explaining which clusters of skills influence the long-term learning utility will help talents better understand and choose their future career paths.

From this intuition, we propose a prototype-based future learning utility approach, which models the long-term utility of talent skill sets as the joint effect of its relevant skill set prototypes. Specifically, SeSRDQN has a skill-oriented prototypical layer to learn the representations of k prototypes, formulated as $P = [p_1, p_2, \dots, p_k]$. We suppose learning a new skill s will result in long-term learning utility $Q_u(s, p_i)$ for a prototype p_i .

Recognizing that a talent's skill set is a multi-faceted combination of various skill aspects, we model the long-term utility for the entire skill set as the weighted sum of the similarity scores

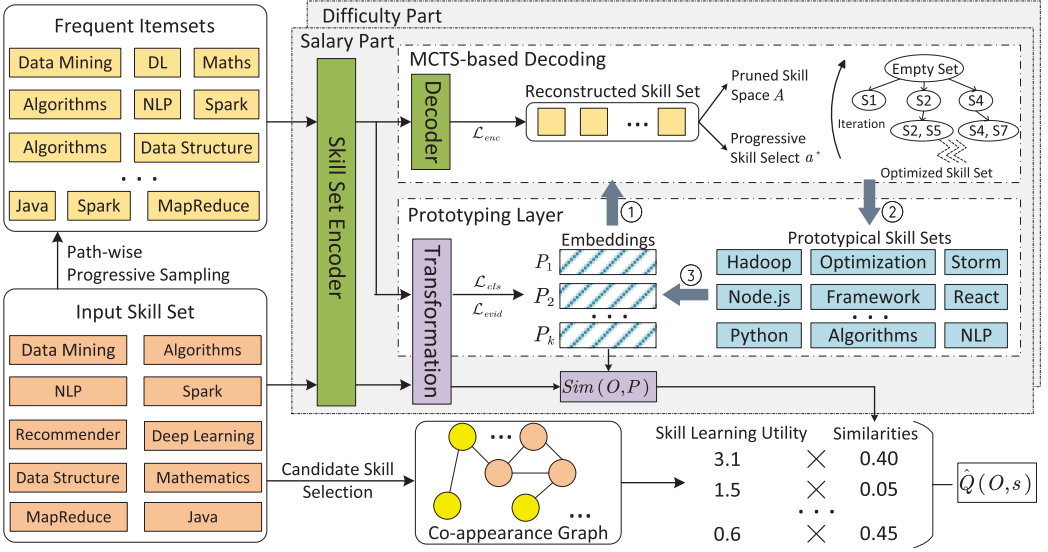


Fig. 5. The architecture of SeSRDQN, which consists of two symmetrical parts for estimating salary and difficulty Q-value separately. We omit the difficulty part for ease of presentation.

between the talent's skill set and the prototypes, formulated as

$$\hat{Q}(O, s) = \sum_{i=1}^k Sim(O, p_i) Q_u(s, p_i), \quad (6)$$

where $Sim(O, p_i)$ represents the similarity score between a skill set prototype and a talent's skill set, formulated by a normalized cosine similarity:

$$Sim(O, p_i) = \frac{1}{2} \left(\frac{z(X_O) \cdot z(p_i)}{\|z(X_O)\|_2 \cdot \|z(p_i)\|_2} + 1 \right), \quad (7)$$

where $z(\cdot)$ represents the transformation function that projects the input skill set or prototypical skill sets into the same embedding space for semantic similarities.

(2) *Frequent Subset-guided Prototyping*. Learning human-understandable and meaningful prototypes for explainable skill recommendation presents many technical and domain challenges. Firstly, the existing prototyping methods are designed for classification tasks, in which the model makes a relative distinction by comparing the input with the prototypes in different classes. However, our task requires learning continuous Q-values for selecting the most promising skill to learn. The simple “this looks like that” comparison is not sufficient for capturing the nuanced relationships between the input skill set and the market skill requirements. This necessitates a more fine-grained prototyping method. Furthermore, unlike the supervised learning paradigm, the deep Q network dynamically generates its training inputs based on the evolving and accumulated state. Directly learning prototypes from the training data may lead to biased and unrealistic prototypes, complicating human understandability and potentially leading to convergence instability.

To model this dynamic in an explainable manner, we consider the multi-faceted nature of talent mastery and propose to constrain the learned prototypes to resemble the decoupled skill groups commonly demanded in market jobs, i.e., frequent subsets in Section 3.2. During the training phase, we extract a collection of frequent subsets from the current input state to regularize the prototype learning process. However, extracting all frequent subsets from the input state is computationally

intensive and yields a large amount of extra data input, thereby reducing training efficiency. To address this, we propose a path-wise progressive sampling strategy, which we can find and sample frequent subsets without going through all the skill combinations. Specifically, we organize the skill sets with a hierarchical graph, as shown in Figure 3, where each node represents a frequent subset, and each edge represents the skill set transition from one state to another. In each bootstrapped step, we identify the end nodes activated by the newly added skill. Combined with the historical frequent subsets, we can obtain the current view of frequent subsets $\mathcal{F}^t$ in the state O^t for sampling. It's worth noting that we apply the experience replay strategy to achieve off-policy training. In particular, we store the recommendation records, denoted by $(O, \mathcal{F}, s, r_{sal}, r_{dif})$, in a memory cache $\mathcal{M}$ and randomly draw a subset, denoted by $\mathcal{D}$, for each training iteration.

To enable learning prototypes semantically similar with frequent subsets, we introduce two regularizations to minimize the Euclidean distance between prototypes and frequent subsets in the embedding space. First, we introduce a clustering regularization to encourage each frequent subset in a batch experience $\mathcal{D}$ to be close to at least one prototype:

$$\mathcal{L}_{cls} = \frac{1}{|\mathcal{D}|} \sum_{O \in \mathcal{D}} \sum_{fr \in \mathcal{F}_O} \min_{j \in [1, k]} \|z(X_{fr}) - z(p_j)\|_2^2, \quad (8)$$

where X_{fr} is the embedding of the extracted frequent subset fr in the batch experience $\mathcal{D}$. We calculate X_{fr} through the skill embedding and sum pooling layer. From the other side, we introduce an evidence regularization to force each prototype to be close to at least one frequent subset:

$$\mathcal{L}_{evid} = \frac{1}{k} \sum_{j=1}^k \min_{fr \in \mathcal{F}_O \wedge O \in \mathcal{D}} \|z(p_j) - z(X_{fr})\|_2^2. \quad (9)$$

Moreover, considering that duplicated prototypes may bring redundant explanations, we introduce a diversity regularization term to penalize the learned prototypes close to each other:

$$\mathcal{L}_{div} = \sum_{i=1}^k \sum_{j=i+1}^k \max \left(0, \frac{p_i \cdot p_j}{\|p_i\|_2 \cdot \|p_j\|_2} - s_{min} \right), \quad (10)$$

where s_{min} is a hyper-parameter to determine whether two prototypes are close or not.

In particular, the prototype embeddings are not directly human-understandable. To enable visualizing the learned prototypes as discrete skill sets, we propose a method to project each prototype into the nearest skill set in the embedding space. In this way, we can conceptually equate each prototype with a semantically meaningful skill set (prototype projection will be detailed in the next subsection). To ensure the learned prototype embeddings could be reconstructed back into skill sets, we introduce a skill set reconstruction loss based on an encoder-decoder structure. Specifically, we set a decoder structure appended with a sigmoid function to reconstruct the skill sets using a binary cross-entropy loss, formulated as

$$\mathcal{L}_{dec} = -\frac{1}{|\mathcal{D}|} \sum_{O \in \mathcal{D}} \sum_{fr \in \mathcal{F}_O} \left(fr \log(\hat{fr}) + (1 - fr) \log(1 - \hat{fr}) \right), \quad (11)$$

where $\hat{fr} = \text{dec}(X_{fr})$ indicates the decoding results of the extracted frequent subset fr , represented by a probability vector over all skills.

To summarize, the training loss of SeSRDQN is formulated as

$$\mathcal{L} = \mathcal{L}_{rew} + \lambda_{freq}(\mathcal{L}_{cls} + \mathcal{L}_{evid}) + \lambda_{dec}\mathcal{L}_{dec} + \lambda_{div}\mathcal{L}_{div}, \quad (12)$$

where λ_{freq} , λ_{enc} , and λ_{div} are hyper-parameters to control the weights of regularization terms.

(3) *MCTS-based Optimization-decoding Iterative Training Procedure*. As mentioned earlier, the learned prototype embeddings are not directly explainable by humans. To make them understandable, we project each prototype to its nearest skill set in the embedding space during the training phase, aligning it with meaningful, human-understandable semantics. However, there are three key challenges: (1) The combinatorial nature of skill sets makes searching for the nearest skill sets difficult due to exponential time complexity; (2) Unlike image and sequence data, each skill has discrete, conceptual, and multi-faceted semantics in different contexts; (3) Traditional prototype projection methods, such as encoder-decoder and beam search, often produce spurious and less representative skill set projection due to the lack of strategic exploration capabilities, thereby affecting the model's explainability.

To overcome these challenges, we develop a hybrid prototype decoding approach based on the decoder and MCTS [56] to guide our skill subset explorations for both accuracy and efficiency considerations. Specifically, we introduce the decoder result as the prior knowledge for encouraging the MCTS to explore the search space with skills that are more relevant to the decoder result. Formally, we define our search tree as follows. Each node in this search tree, denoted as N_i , represents a combinatorial skill set. As a root node, an initial state N_0 is defined as an empty skill set ϕ . Each edge represents the operation of adding a new skill s . We define that the node state N_j is obtained by adding action s_j from its parent node N_i , where $N_j = N_i \cup \{s_j\}$. Considering the large action space for expanding a node, we leverage the co-occurrence frequency of the decoder result to help prune most irrelevant actions. Specifically, we define the set of expanding actions as $A_i = \{s\}$, s.t. $\exists fr: (s \in fr) \wedge (|\hat{g} \cap fr| \geq n)$, where fr is one of the frequent subsets, $\hat{g}$ is the decoder result for a learned prototype, and n is a threshold reflecting the confidence level of $\hat{g}$.

Formally, we define four fundamental variables in our MCTS algorithm as follows:

- $C(N_i, s_j)$ is the number of counts for selecting the action s_j for node N_i .
- $W(N_i, s_j)$ denotes the total reward for all (N_i, s_j) visits.
- $Q(N_i, s_j) = W(N_i, s_j)/C(N_i, s_j)$ denotes the average reward for all (N_i, s_j) visits.
- $R(N_i, s_j)$ is the immediate reward of choosing s_j on N_i . Here, we measure this reward by a difference-of-logarithms function $\log((\|z(X_{N_i}) - z(p)\|_2^2 + 1)/(\|z(X_{N_i}) - z(p)\|_2^2 + \epsilon))$, where X_{N_i} is the state embedding of N_i and p is the given prototype for searching.

In each rollout, the MCTS algorithm will traverse from the root node N_0 to a leaf node by following the pre-defined action selection rule. We restrict the state size of leaf nodes to be in the range $[N_{min}, N_{max}]$. Formally, we define the action selection strategy as follows:

$$a^* = \arg \max_{a_j} Q(N_i, s_j) + U(N_i, s_j) + \frac{H_j}{1 + C(N_i, s_j)}, \quad (13)$$

$$U(N_i, s_j) = \lambda R(N_i, s_j) \frac{\sqrt{\sum_k C(N_i, s_k)}}{1 + C(N_i, s_j)}, \quad (14)$$

$$H_j = \begin{cases} \tau & \text{for } \{N_i\} \cup \{s_j\} \subseteq \hat{g} \\ 0 & \text{for } \{N_i\} \cup \{s_j\} \not\subseteq \hat{g} \end{cases}, \quad (15)$$

where H_j is a progressive bias and λ is a hyper-parameter to control the tradeoff between exploration and exploitation. For all the child nodes of a newly visited node with $C = 0$, this threshold value τ would encourage the selection of nodes that have high relevance with the decoder result. Thereafter, the impact of this bias continually decreases with the increment of the total visiting count. At early stages, the selection strategy prefers to choose nodes with low counts or high relevance to explore possible combinations. Then it will converge to an optimized selection strategy when numerous rollouts are performed.

Algorithm 3: Model Parameter Training for SeSRDQN

```

1: procedure MODEL_TRAINING( $epoch, \Phi, \Phi^-$ )
2:   for  $it = 1 \rightarrow epoch$  do
3:      $O^1 \leftarrow$  draw a skill set from  $\mathcal{I}$ ;
4:      $\mathcal{F}^1 \leftarrow$  extract frequent subsets from  $O^1$ ;
5:     for  $t = 1 \rightarrow T$  do
6:        $C \leftarrow$  find candidate skills of state  $O^t$ ;
7:       Sample  $s \sim \mu(\cdot, O^t; \Phi^-)$  from  $C$ ;
8:       Find incremented frequent subsets  $\mathcal{F}_{new}$  activated by  $s$  from  $O^t$ ;
9:       Sample  $\mathcal{F}_{sample}$  from  $\mathcal{F}^t$ ;
10:       $\mathcal{M} \leftarrow \mathcal{M} \cup (s, O^t, \mathcal{F}_{sample}, f_{sal}(s, O^t), f_{dif}(s, O^t))$ ;
11:       $O^{t+1} \leftarrow O^t \cup \{s\}$ ;
12:       $\mathcal{F}^{t+1} \leftarrow \mathcal{F}^t \cup \mathcal{F}_{new}$ ;
13:      for  $i = 1 \rightarrow N_{batch}$  do
14:         $(s, O, \mathcal{F}, r_{sal}, r_{dif}) \leftarrow$  draw a training sample in  $\mathcal{M}$ ;
15:         $C \leftarrow$  find candidate skills of state  $O \cup \{s\}$ ;
16:         $s^* \leftarrow \arg \max_{s' \in C} \hat{Q}(s', O \cup \{s\}; \Phi^-)$ ;
17:         $y_{sal} \leftarrow r_{sal} + \gamma \hat{Q}^{sal}(s^*, O \cup \{s\}; \Phi^-)$ ;
18:         $y_{dif} \leftarrow r_{dif} + \gamma \hat{Q}^{dif}(s^*, O \cup \{s\}; \Phi^-)$ ;
19:         $d\Phi \leftarrow d\Phi + \frac{\partial \mathcal{L}_{sal}(s, O, \mathcal{F}, y_{sal}, \Phi)}{\partial \Phi} + \frac{\partial \mathcal{L}_{dif}(s, O, \mathcal{F}, y_{dif}, \Phi)}{\partial \Phi}$ ;
20:         $\Phi \leftarrow \Phi - \tau d\Phi$ ;
21:        if  $it \bmod IT_{update} == 0$  then
22:          Update target network  $\Phi^- \leftarrow \Phi$ ;
23:   return  $\Phi, \Phi^-$ 

```

Algorithm 4: Overview of SeSRDQN Training

Input: $\mathcal{I}$: the set of initial skill sets; T : number of steps in an episode; N_{batch} : batch size for training; IT_{update} : update period; IT_{train} : training iterations; $IT_{finetune}$: fine-tuning iterations; τ : learning rate;

Output: parameter of the trained model;

```

1: Initialize model parameters  $Q_{net}$ ;
2: Initialize a global empty memory set  $\mathcal{M}$ ;
3:  $Q_{net}, Q_{target} \leftarrow$  MODEL_TRAINING( $IT_{train}, Q_{net}, Q_{target}$ );
4:  $P_{sal}, P_{dif} \leftarrow$  Performing prototype decoding;
5: Replace the prototype layer in  $Q_{net}$  with  $P_{sal}, P_{dif}$  and set them non-trainable;
6:  $Q_{net}, Q_{target} \leftarrow$  MODEL_TRAINING( $IT_{finetune}, Q_{net}, Q_{target}$ );
7: return  $Q_{net}$ ;

```

In the back-propagation phase, the statistics of all nodes in the selected path are updated by the following rule:

$$C(\mathcal{N}_i, s_j) = C(\mathcal{N}_i, s_j) + 1, \quad (16)$$

$$W(\mathcal{N}_i, s_j) = W(\mathcal{N}_i, s_j) + R(\mathcal{N}_i, s_j). \quad (17)$$

At last, we choose the leaf node that has the highest similarity with the given prototype.

This hybrid prototype decoding method sets the foundation for improving prototype decoding. By incorporating it into the iterative learning process, we ensure that the model not only optimizes skill recommendations but also explains its choices clearly. The model parameter training algorithm of SeSRDQN is summarized in Algorithm 3 and the whole training procedure is shown in Algorithm 4. It covers the *frequent subset-guided prototype learning process*, *prototype decoding*, and

model fine-tuning process for enhanced performance. We first initialize the model parameters and an empty memory set $\mathcal{M}$ to store the training data. Then we perform the model training procedure for IT_{train} iterations. In each iteration, we randomly draw a skill set from the initial skill set $\mathcal{I}$. Then we simulate the recommendation process for T steps and store the corresponding records in $\mathcal{M}$. After that, we randomly draw a subset $\mathcal{D}$ from $\mathcal{M}$ and perform the model parameter training with the standard DQN algorithm. In the training phase, we employ the experience replay strategy to achieve off-policy training. Specifically, we store the data records, denoted by $(O, \mathcal{F}, s, r_{sal}, r_{dif})$, in a memory cache $\mathcal{M}$ and randomly draw a batch experience $\mathcal{D}$ for each training iteration. After the model parameter training, we perform the prototype decoding to replace the prototype layer in Q_{net} with the most relevant skill set representations and set them non-trainable. Finally, we fine-tune the model parameters for $IT_{finetune}$ iterations to further improve the model performance.

4.4 Complexity Analysis

The complexity of training a skill recommendation model primarily arises from the exponential number of possible skill combinations, leading to high complexity in state transitions and estimations. To address this, we have designed several strategies to minimize training complexity. First, the candidate selector reduces the number of skills considered for recommendation to size N_c , significantly lowering the time complexity for Q-value estimation. It's noteworthy that the skill graph is constructed offline, incurring only a one-time build cost. Second, in the frequent subset-guided prototype learning process, we adopt a path-wise progressive sampling method to extract frequent subsets without going through all the skill combinations. This method enables linear subset sampling by organizing the historical state as a hierarchical graph and sequentially updating it, with a complexity of $O(|X_O| + T)$. Third, the MCTS-based prototype decoding algorithm efficiently searches for the nearest skill sets for prototype projection. The introduced progressive bias integrates the frequent subset information to guide the search process to more relevant skill combinations, significantly reducing the need for extensive exploration. The efficiency of this algorithm is validated in Table 3. With these strategies, the overall complexity of SeSRDQN is significantly reduced, making it reasonable and practical for this skill recommendation task.

5 Experiments

In this section, we carry out experiments designed to address the following seven critical questions:

- Q1: How does SeSRDQN perform in terms of salary increase and learning difficulty?
- Q2: How do the different components of SeSRDQN influence the recommendation performance?
- Q3: How do the parameters of SeSRDQN influence the recommendation performance?
- Q4: How efficient is our SeSRDQN for prototype projection?
- Q5: How well does our SeSRDQN identify the globally representative skill sets?
- Q6: How well does our SeSRDQN explain the recommended learning path based on quantitative Q-value estimation?
- Q7: How does the integration of pre-trained models influence the recommendation performance?

5.1 Experimental Setup

In this section, we evaluate the performance of our approach with extensive experiments on the real-world job posting dataset [61].

5.1.1 Dataset. The descriptions of the real-world job posting dataset can be found in Section 2.1. We conducted experiments on IT-related jobs since they are representative skilled occupations.

We first applied *Jieba* to perform word segmentation on the job descriptions and filtered the n-grams with high appearing frequencies. After manually selecting and merging the words, we got 987 IT-related job skills. To reduce noise, we filtered out the job postings containing less than 10 skills and got 805,182 job postings in total.

5.1.2 Frequent Itemset. By regarding each job posting as a combination of skills, we mined frequent skill sets with FPGrowth. Specifically, we empirically regarded a skill set as frequent when its support (i.e., θ_{dif}) is larger than 0.005 in this work. Then, we filtered the sets containing no larger than 10 skills and got 374,616 frequent skill sets in total.

5.1.3 Initial Set Construction. We sampled a set of initial skill sets from the job posting data to train and validate our model. Due to the market demand differences, jobs are not evenly distributed in the market. Therefore, we iteratively sample the sets to balance the job duty distribution. Specifically, in each iteration, we measured the total skill appearance frequency of a job posting J^j as $\sum_{s \in J^j} \#(s)$, where $\#(s)$ denotes the number of the current selected initial sets that contain the skill s . Then, we randomly selected 100 samples from the 50,000 job postings that have the lowest frequency empirically. This process was repeated until we got 150,000 skill sets. From these sets, we randomly picked 5,000 test samples and used the rest as training samples.

5.1.4 Configurations. Unless specified, we used a set of default settings in our experiment. Specifically, for the environment, the parameter w , and threshold θ_{sal} are set to be 5 and 0.9, respectively, and the salary estimator averages the top-100 job salary. For the recommender, we set the candidate pool size (i.e., N_c) to be 100, the difficulty importance factor (i.e., α) to be 0.1, and the discount factor (i.e., γ) to be 0.8. As for the loss function hyper-parameters, we set $\lambda_{freq} = 0.01$, $\lambda_{div} = 0.1$, and $\lambda_{enc} = 0.1$. We trained the model with RMSProp optimizer. The weights are initialized with glorot normal initializer [19]. All the MLP layers were composed of three fully connected layers with 256 hidden units, which is the same as the embedding dimension. For Monte Carlo Tree Search, we set $N_{min} = 2$, $N_{max} = 9$, and a small progressive bias $\tau = 0.1$. The code of the experimental evaluation, including the baselines and our proposed method, is released on Github.¹

5.1.5 Baseline Methods. Following the real-world skill learning strategies, we constructed several baseline methods, including:

- *Sampling Methods.* We picked from the candidate pool according to (1) uniform distribution and (2) normalized appearing frequency.
- *Large Language Model (LLM) Methods.* We leveraged LLM to help select the most suitable skill learning path. We used three prevalent LLMs, including Meta Llama3-8B [64], QWen2-7B [5], and GPT-4 [1].
- *Greedy Methods.* We recommended the candidate skill which achieves (1) the highest instant salary, (2) the lowest learning difficulty, and (3) the highest instant reward.
- *Model-based Methods.* We trained **Deep Neural Network (DNN)** models that take the state as the input and predict the instant utilities of each candidate skill. Specifically, the utilities include the appearing probability (“Action DNN”), instant salary benefit (“Salary DNN”), learning difficulty (“Difficulty DNN”), and reward (“Reward DNN”). We constructed training data from the job postings to train these models and recommend the skill with the highest prediction.

¹<https://github.com/yangji721/SeSRDQN>

Table 2. Performance Evaluation for Skill Recommendation

Type	Method	4-Step	6-Step	8-Step	10-Step	Salary 12-Step	14-Step	16-Step	18-Step	20-Step	Difficulty
Random	Uniform Frequency	1.87	1.94	2.03	2.11	2.20	2.29	2.39	2.48	2.59	3.97
		2.09	2.28	2.49	2.73	3.00	3.30	3.63	3.97	4.39	3.11
LLM	Llama3	3.24	4.28	5.52	6.55	7.85	9.15	10.38	11.44	12.38	2.05
	QWen2	2.79	3.75	4.82	5.84	7.03	8.31	9.58	10.55	11.36	2.11
	GPT-4	3.48	4.97	6.38	7.87	9.26	10.45	11.61	12.77	13.66	1.58
Greedy	Salary	7.68	11.17	14.81	18.66	22.53	26.16	29.32	31.92	34.11	1.24
	Difficulty	4.82	6.93	9.38	11.98	14.86	17.97	21.07	23.89	26.28	0.29
	Reward	<u>7.92</u>	11.77	15.84	19.92	23.84	27.35	30.36	32.90	35.09	0.79
Utility Model	Action DNN	3.67	4.67	5.96	7.50	9.57	12.33	15.07	17.60	20.09	0.42
	Salary DNN	3.47	4.45	5.57	6.70	7.93	9.06	10.22	11.27	12.28	1.90
	Difficulty DNN	3.12	4.08	5.35	7.04	9.20	11.80	14.66	17.71	20.72	0.42
	Reward DNN	3.06	3.92	4.87	6.02	7.50	9.18	10.95	12.69	14.38	1.78
Utility + Exploration	Salary DNN	6.42	9.42	12.65	15.96	19.24	22.37	25.24	27.93	30.33	1.23
	Difficulty DNN	4.72	6.80	9.18	11.78	14.66	17.83	20.99	23.89	26.38	0.32
	Reward DNN	6.86	10.52	14.71	19.16	23.56	27.50	30.82	33.49	35.69	0.64
SeSRDQN	SRDQN	7.03	11.93	17.95	23.57	27.69	31.16	34.07	36.33	38.24	0.66
	SRDQN + i-DQN	3.64	5.62	9.69	15.27	21.08	26.31	30.55	33.63	35.53	0.67
	SRDQN + ProSeNet	3.23	5.39	10.04	15.38	20.34	25.34	29.29	32.82	35.93	0.60
	SeSRDQN(single)	3.04	4.97	9.80	15.41	21.19	26.78	31.52	34.49	36.47	0.68
	SeSRDQN	6.56	10.57	16.12	22.10	26.92	31.28	34.12	36.15	38.25	0.65

The best results are highlighted in *bold and underlined*, and the second-best results are in *bold*.

—*Model-based Methods Enhanced with Exploration.* We utilized the exploration strategy of SeSRDQN to actively acquire more training samples, and thus strengthen “Salary DNN,” “Difficulty DNN,” and “Reward DNN.” Specifically, we utilized the ϵ -greedy strategy to sequentially sample actions and stored and replayed these records to train the models.

5.2 RQ1: Performance Evaluation

For each testing sample, we recommended 20 steps of skills with the baselines and our SeSRDQN.

5.2.1 Baseline Performance Analysis. In Table 2, we show the average difficulty and salary at different steps. It can be observed that the sampling methods perform poorly due to the absence of adaptive refinement and contextual learning adjustments. This suggests that indiscriminate skill acquisition is not effective for career development. The LLM methods also have a performance gap compared with the greedy methods and utility models. This is mainly because the LLM methods are not specifically designed for the skill recommendation task and thus may not fully capture the intricate skill relationships. Difficulty Greedy and Salary Greedy assure one-sided high utility but lose performance on the other perspective. Reward Greedy achieves good performance by capturing both salary and difficulty utilities. However, it should be noticed that they have high time complexity and are sensitive to data noise. For example, if a company posts abnormally highly paid jobs, these methods will overestimate their required skills’ salary benefit without noticing the anomaly. Compared with the greedy methods, the utility models also greedily take actions, but they approximate the utilities with model parameters. Though losing some accuracy in value estimation, they are faster and can better resist noise. However, we can observe from Table 2 that it is difficult to train these models with traditional supervised learning methods with limited job posting data. Therefore, we find that an active exploration strategy can help keep the greedy-based models from over-fitting and thus raise the model performance significantly.

5.2.2 SeSRDQN Performance Analysis. There are several observations. First, SRDQN outperforms all the baseline models by considering the combined factors of salary and learning difficulty, proving the effectiveness of our proposed framework. Second, while SeSRDQN provides a comprehensive explanation for the recommendation results, it still achieves comparable performance with SRDQN. This shows that SeSRDQN can effectively capture the skill relationship and thus maintain a competitive performance. Third, even when adopting the same exploring strategy, SeSRDQN still outperforms the utility models, which demonstrates the effectiveness of SeSRDQN in terms of sustainable recommendations. Fourth, greedy-based methods perform better than SeSRDQN at the first several steps. As we have discussed, their inborn advantages come from the exhaustive requests for instant rewards. But after eight steps, SeSRDQN outperforms all these methods, implying that the long-sighted modeling makes up the accuracy loss of parameter approximation. Moreover, SeSRDQN generally achieves low learning difficulty. Though there are baseline methods that achieve lower difficulty, they all show a great loss in salary benefits. SeSRDQN no doubt has a great advantage over them because it assures high salary benefits when keeping low learning difficulty.

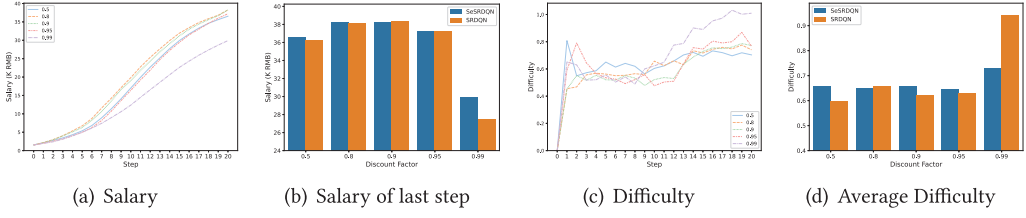
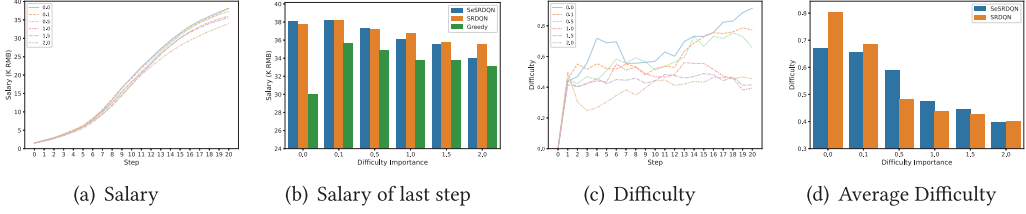
5.3 RQ2: Ablation Studies

We disable or replace some parts of the network to reveal the effects of its components. We first disable the multi-task structure in SeSRDQN and let the model only estimate the mixed Q-value function, denoted by “SeSRDQN(single)” in Table 2. The performance decreases a lot at the early steps of recommendations, revealing that the separate Q-value estimation not only raises the recommendation explainability but also can increase the fitting ability of the model by knowledge sharing between the two tasks. Then, we replace SeSRDQN with variants of SRDQN framework incorporated with the dual structure (denoted by “SRDQN”) and two existing explainable approaches, including i-DQN [4] and ProSeNet [42]. Both explainable baselines suffer a performance gap relative to the black-box counterparts, highlighting the tradeoff between model explainability and performance. This is mainly because the unordered nature of skills and the intertwined semantic relationships between skill sets complicate the direct application of these methods. In contrast, SeSRDQN achieves comparable or even superior salary benefits to SRDQN after the 12th step, while also presenting lower learning difficulty. This demonstrates the effectiveness of our designed frequent subset-guided prototype learning method in disentangling skill relationships, thus providing an explainable and high-performing recommendation.

5.4 RQ3: Parameter Experiment

We perform parameter experiments on four important parameters that could influence the recommendation of SeSRDQN, which are discount factor γ , difficulty importance α , candidate pool size N_c , and prototype number k . Here we adjust their values to show their effects. Specifically, for each factor’s influence on salary or difficulty, we draw the corresponding figures to show the overall trend. Since the scale from the first step to the last step may be large, we further draw figures to show the salary performance of the last step and the average skill-learning difficulty.

5.4.1 Discount Factor. In Figure 6, we show the salary and difficulty for models trained with different discount factors γ in the range of (0.99, 0.95, 0.9, 0.8, 0.5). It can be observed that the trends of salary increase and difficulty are similar for most discount factors. Generally, a larger value leads to lower short-term but higher long-term utility. For example, when $\gamma = 0.95$, the model achieves a relatively low salary and high difficulty in the first few steps but soon catches up with other settings. It is notable that when the value is too large (i.e., $\gamma = 0.99$), the model fails to converge to a proper Q-value estimation. It can be observed that SeSRDQN achieves comparable performance

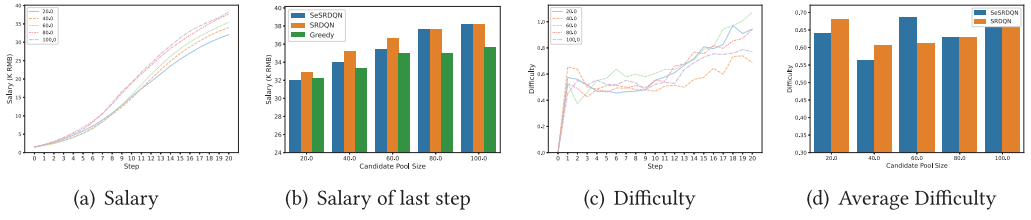
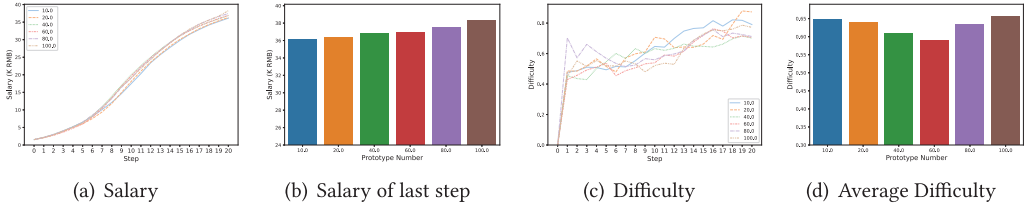
Fig. 6. Influence of discount factor γ .Fig. 7. Influence of importance factor α .

with SRDQN when γ in (0.8, 0.9, 0.95) and has better model convergence when γ is too large (0.99) or small (0.5).

5.4.2 Difficulty Importance. In Figure 7, we vary the difficulty importance factor α in the range of (0, 0.1, 0.5, 1, 1.5, 2). It can be observed that α balances the tradeoff between salary and difficulty. A larger value significantly decreases the learning difficulty while bringing down the salary benefit, especially in the long term. Specifically, when α is larger than 0.5, the difficulty remains low for the whole recommendation. In contrast, when γ is smaller, the learning difficulty increases as the recommendation gets longer. However, Figure 7(b) implies that laying more importance on difficulty has a negative influence on long-term salary benefits. Figure 7(b) also reveals that SeSRDQN always outperforms greedy methods and has a strong ability to estimate learning utilities.

5.4.3 Candidate Pool Size. In Figure 8, we vary the candidate pool size N_c in the range of (20, 40, 60, 80, 100). For better understanding, in Figure 8(b), we also visualize the salary of the last step achieved by Reward Greedy and SRDQN as comparisons. It can be observed that the raising candidate pool size affects little on the learning difficulty but has a positive influence on long-term salary benefits. Generally, bigger N_c leads to higher performance. It is notable that increasing candidate pool size affects little on the baseline model performance. The reason is that the baseline models only consider the instant utility and high instant utility. No matter salary or difficulty, the higher utility will be brought by highly relevant skills, which appear at the top of the candidate pool. In contrast, SeSRDQN and SRDQN can find the skills that are less relevant to the current skill set but potentially bring high long-term benefits. Therefore, a larger candidate pool can increase long-term performance by enlarging the search space of SeSRDQN and SRDQN.

5.4.4 Prototype Number. In Figure 9, we show the salary and difficulty for models trained with prototype number k in the range of (10, 20, 40, 60, 80, 100). Our results reveal that increasing k positively affects model performance, particularly in terms of salary. Larger k can enrich the model expressiveness to estimate Q values, thereby improving recommendation quality. Notably, even with a smaller k , our model outperforms the baselines, highlighting its effectiveness. As the salary potential increases, we also observe a natural rise in learning difficulty, aligning with the expectation that higher-paying jobs are more challenging to attain. An interesting observation is

Fig. 8. Influence of pool size N_c .Fig. 9. Influence of prototype number k .

that when k is below 40, there is an increase in learning difficulty accompanied by a decrease in salary. This trend suggests that SeSRDQN starts to lose some expressiveness when k is too small. Indeed, k represents the tradeoff between model expressiveness and explanation complexity. By considering this, we can adjust k for different purposes in real-world applications or incorporate some existing prototype pruning methods [20, 53] to simplify the explanations.

5.5 RQ4: Evaluation of Prototype Projection

Prototype projection is a critical component of our SeSRDQN, as it conceptually equates each prototype with a discrete skill set for human understanding. The selected prototypes should be representative of the frequent skill set space, which is more intuitive and human-readable. Moreover, searching the nearest skill sets should be efficient to ensure the model's scalability. Therefore, we provide a quantitative evaluation to compare the fidelity and efficiency of prototype projection with the established baseline methods in Table 3: (1) the decoder approach [28] that directly decodes the prototypes to the skill set space; (2) the selective iteration method by [11], which identifies the nearest ones by exploring all possible frequent skill sets; (3) the greedy-based beam search technique from [42]; and (4) the straightforward MCTS approach by [80].

We utilize a difference-of-logarithms function [11] to measure the fidelity score, which represents the similarity between the projected skill set embedding and the original prototype embedding. It is defined as $\log((\|z(X_O) - z(p)\|_2^2 + 1)/(\|z(X_O) - z(p)\|_2^2 + \epsilon))$, which provides a monotonically decreasing function of the Euclidean distance. It can be observed that our method achieves a notably high fidelity score while maintaining a reasonable time cost. Specifically, the selective iteration method [11] over all possible frequent skill sets is too time-consuming to be applied here since the frequent skill set space is still too large. The beam search [42] method and decoder method [28] are much faster than the selective iteration method but suffer a performance gap. The MCTS method [80] suffers from a significant efficiency degradation due to the extremely large search spaces. These results show that our method can effectively project the prototypes from the embedding space to the semantic skill sets with high efficiency and fidelity.

We further provide a case study to demonstrate that our method effectively captures the semantic essence of frequent skill sets and provides representative prototypes. For instance, the

Table 3. Efficiency Studies of Different Prototype Projection Methods

Methods	Decoder	Selective Iteration	Beam Search	MCTS	Our Method
Fidelity	0.569	1.351	1.193	1.997	1.989
Time	0.1 s	488.7 s	32.0 s	1,273.5 s	61.9 s

Table 4. Example Prototypes of Skill Sets in Terms of Salary Increase and Learning Difficulty

Module	Prototypes	Global Influence
#Salary	Storage, Oracle, Hibernate, Spring, MyBaits	0.304
	Tomcat, Shell, Socket, Software Development	0.292
	Operating System, Shell, Hibernate	0.288
	Linux, Programming, Testing, Perl, Scripting	0.270
	Frontend Development, HTTP, Web	0.252
#Difficulty	Performance, MySQL, Database, Storage	0.581
	Firewall, Programming, Software Development	0.570
	Testing Tools, Software Testing, O&M, Automation Testing	0.567
	Frontend Development, Web, Angular	0.565
	ML, R Language, Statistics	0.565

prototype generated by our method (“*Performance*,” “*MySQL*,” “*Database*,” “*Storage*”) falls within globally frequent itemsets and explicitly captures database management abilities. In contrast, the decoder result (“*Performance*”) is overly simplistic and could be misinterpreted, as it also implies high-performance computing in other contexts. Additionally, the decoder method’s sensitivity to parameter settings can result in unstable and spurious outputs. The greedy-based beam search result (“*Database*,” “*Frontend Development*,” “*Frontend Optimization*,” “*MySQL*,” “*Python*”) includes spurious skill semantics, making it less semantically dense and further complicating user comprehension. By efficiently encouraging the formation of semantically dense skill sets, our method captures more comprehensive and representative skill sets. To further analyze the impact of prototype projection on model explainability, we present detailed human-centered user study results in Section 5.9.

5.6 RQ5: Quantitative Analysis on Skill Set Prototypes

SeSRDQN captures the inherent impact of various combinations of prototypical skill subsets on salary increase and learning difficulty.

5.6.1 Global Influence Analysis. We average the weight contributions Q_u of all the skills in the dataset to obtain the global influence of each prototype. It indicates the averaged importance of each prototype in the model reasoning process. In Table 4, we list the skill set prototypes with top-5 weight contributions Q_u averaged over all the skills.

There are several observations. Firstly, the prototypes used for estimating salary and difficulty Q-values exhibit distinct characteristics. As for salary, prototypes lean towards specific, job-related skills like “*Spring*,” “*Tomcat*,” and “*HTTP*,” indicating that nuanced, role-specific skills. Conversely, difficulty-oriented prototypes are shaped by more general, foundational skill combinations, suggesting that fundamental knowledge forms the basis of learning difficulty. It conforms our method’s capability to effectively discern key patterns that impact long-term effects on salary and difficulty utilities. Furthermore, there exists the strong interrelation of skills within each prototype. These

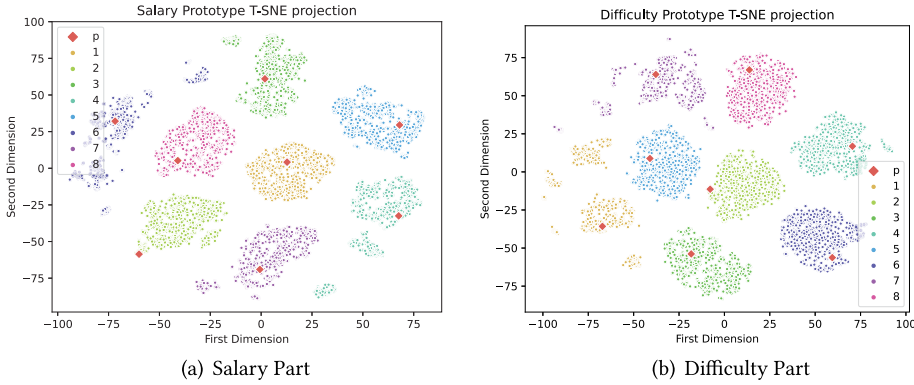


Fig. 10. tSNE visualization on the dataset. The clusters of skill itemsets are distinguished by different colors.



Fig. 11. Initial sets for case studies.

prototypes not only cluster skills but also reflect coherent, semantically meaningful skill combinations that align with the market demand. These results demonstrate our method's effectiveness in discovering critical skill sets that influence long-term career trajectories.

5.6.2 Prototype Visualization. In Figure 10, we carry out the visualization experiments on the frequent skill sets and prototype representations using the tSNE method. For better pattern discovery, we separately performed k -means over the frequent skill sets and prototypes and visualized the clusters with different colors. We have the two important observations. Firstly, the frequent skill sets form several distinct clusters, indicating that skill combinations in the market exhibit decoupled characteristics. Moreover, these prototypes are distributed across different clusters, effectively capturing the semantic essence of frequent skill sets. This distribution showcases the prototypes are representatives of the variety and nuances of skills valued in the market.

5.7 RQ6: Case Study on Recommendation Process

To get more insight into the recommendation process of SeSRDQN, we did a case study with two different skill sets, which are shown in Figure 11. It can be observed that the first skill set is about frontend development, and the second mainly contains generic programming skills, which may be owned by a CS graduate student. To simplify the analysis, we set $N_c = 30$, $k = 30$, $\lambda_{freq} = 0.01$, $\lambda_{div} = 1.0$, $\lambda_{enc} = 0.1$ in this part.

5.7.1 Candidate Skill Analysis. In Figure 12, we show the candidate skills of each skill set, where the word size represents their relevance with the initial set (defined in Section 4.2). It can be

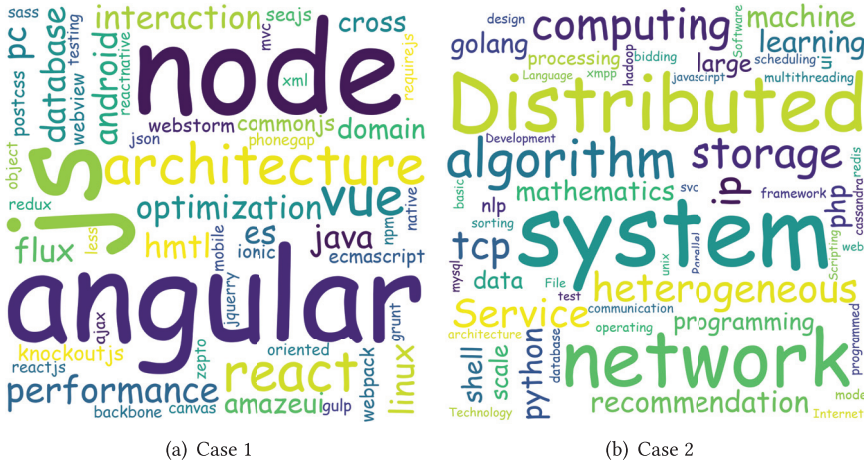


Fig. 12. Wordcloud of the candidate skills.

observed that the candidate skills do have strong semantic connections with the skill set, which intuitively can bring high salary benefits with an acceptable learning difficulty. For the first case, most candidate skills are deeply related to the current job duty, among which “Node.js,” “Angular.js,” and “Architecture” have the highest relevance, leading to low learning difficulty. Besides, there is no doubt that these relevant skills can raise the talent’s competency in Web development-related jobs. For the second case, since the skill set is more generic and fundamental, its candidate skills are not limited to specific job duties, which include skills in algorithm engineering, testing, Internet services, distributed computing, and so on. Indeed, for a CS graduate, their solid fundamental skills can help them quickly adapt to various job positions.

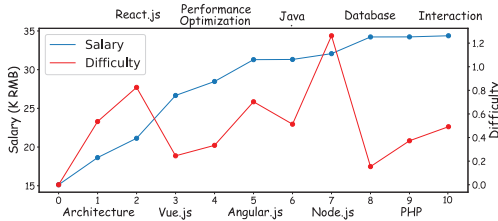
5.7.2 First Step Recommendation. SeSRDQN makes recommendations according to the long-term utility estimation of each candidate skill. In Table 5, we list the salary Q-value, difficulty Q-value, instant salary, and instant learning difficulty of the top 10 skills recommended by SeSRDQN. It can be observed that SeSRDQN can provide reasonable and long-sighted learning choices. Generally, the recommended skills all have considerable long-term and short-term advantages. For example, skills like “Architecture” and “React.js” in the first case have high instant benefits. Indeed, instant reward plays an important role in the total reward, i.e., the discounted summation. Some skills have a relatively lower short-term benefit, but SeSRDQN finds them to have a high long-term benefit, such as “Math” in the second case. This is reasonable because many complicated job duties require the talent’s math foundation. Besides, though some skills have relatively lower salary benefits, they are recommended if they are easy to be learned, such as “Database” in the first case. Similarly, some relatively difficult skills will be recommended for high salary benefits, such as “Recommender System” in the second case. These results show SeSRDQN to comprehensively balance the multi-perspective benefits instead of providing single-sided recommendations.

5.7.3 Sequential Recommendation Visualization. For each case, we recommended 10 skills with SeSRDQN. Specifically, at each step, SeSRDQN chooses the skill with the highest Q-value. In Figure 13, we show the recommended skills and plot the salary and learning difficulty of each step. There are several observations. First, SeSRDQN can recommend sustainable and realistic skill-learning paths according to the situation. For example, SeSRDQN suggests the first case to improve

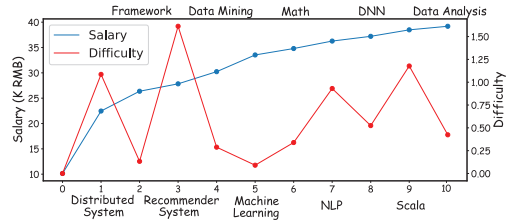
Table 5. Top-10 Candidates at the First Step

Case	Candidate	SalQ	DiffQ	Sal	Diff
#1	Architecture	14.34	2.31	18.64	0.54
	React.js	12.24	2.56	17.35	0.83
	Vue.js	11.96	2.56	16.83	0.65
	Performance Optimization	10.73	2.84	17.41	0.33
	Angular.js	9.47	3.09	15.34	1.42
	Java	9.01	2.71	15.22	0.59
	Android	8.67	2.76	15.29	0.56
	Linux	7.28	3.16	15.13	1.02
	Node.js	6.99	3.18	15.19	1.83
	Database	6.38	2.88	15.16	0.63
#2	Distributed System	13.37	3.49	22.46	1.09
	Data Mining	12.52	2.80	11.28	0.29
	Math ^a	11.56	3.00	10.97	0.37
	Framework	11.48	2.91	13.65	0.19
	Recommender System	11.04	4.23	11.81	1.61
	NLP	10.78	4.08	11.50	1.50
	Unix	8.53	4.23	11.81	0.21
	Scala	8.37	3.94	10.14	1.18
	PHP	8.03	4.07	12.06	1.05
	Web	7.91	3.30	10.24	0.82

^aMath represents essential math skills for engineers.



(a) Case 1



(b) Case 2

Fig. 13. Salary and difficulty of the sequential recommendation for case studies.

Web development skills since the talent has already been familiar with this field. In the second case, while the talent has multiple career directions, SeSRDQN suggests learning towards jobs related to data analytics and distributed computing. Considering that high-performance computing jobs have been highly paid in recent years, this result demonstrates the strong ability of SeSRDQN to seize market demand. Second, both paths have significant salary increases and acceptable difficulty levels and share similar patterns. In the first several steps, the learning difficulty is generally lower, and the salary increases fast. But the increased speed slows down soon, which represents the glass ceiling phenomenon in the talent career. In order to break the ceiling, the talents need to surmount learning difficulties, such as “Node.js” in the first case and “Recommender” in the second case. Despite that learning some skills might not bring a noticeable salary increase immediately, these skills are crucial for a talent career in the long term.

Table 6. The Recommended Learning Paths

Method	Path
Salary Greedy	Algorithm Basics→Search Engine→ML→Recommender Systems→Data Mining→NLP → Algorithm Design→Distributed Systems→Large-scale Computing→Spark
Reward Greedy	Algorithm Basic→Search Engine→Framework→Distributed System→Data Mining→ ML→NLP→Recommender System →Large-scale Computing→Spark
Difficulty Greedy	Unix→Framework→MySQL→Database→Data Mining→ML→Shell→Spark → HBase →Math
Action DNN	Framework→Testing→Database→Web→JavaScript→MySQL→SQL→Redis→TCP→ Automation
Salary DNN	Framework→Internet→Web→ <i>Algorithm Design</i> →JavaScript→HTML→PHP→Complexity→ CSS → <i>Testing</i>
Reward DNN	Framework→Network→MySQL→Web→Scripting language→PHP→TCP/IP→HTML→System Optimization →Storage
Difficulty DNN	JavaScript→Testing→Shell→MySQL→Database→Web→SQL→Framework→Redis→ <i>Data Min- ing</i>
SeSRDQN	<i>Distributed System</i> → <i>Framework</i> → <i>Recommender System</i> → <i>Data Mining</i> →ML→ Math→NLP→DNN→Scala→ <i>Data Analysis</i>

It should be noticed that though we show the path of learning the top recommendation at each step, the talent can decide their own learning path from candidate skills in Section 5.7.2. This case study shows the ability of SeSRDQN to give skill learning simulation, which helps talents to plan their career. With this simulation, talents will be able to smoothly change their expertise, and further change their career direction according to their personal preferences.

5.7.4 Learning Path Comparison. We compared the recommended learning paths of different models for the second case (i.e., fundamental skills), which are shown in Table 6. Generally, the recommended learning path varies for these methods. Specifically, salary-oriented methods recommend advanced skills for obtaining high-salary benefits. However, they fail to design a proper learning path to master these difficult skills sustainably. For example, Salary DNN fails to generate a unified learning direction, which recommends different kinds of skills, including algorithms, database, Web development, and network programming. Learning too many irrelevant skills could significantly increase the skill-learning difficulty for talents. Second, difficulty-oriented methods tend to start from the fundamental skills and gradually recommend more advanced skills. Their learning paths are more sustainable. For example, skills like “MySQL” and “Database” make the talent easier to learn “Data Mining” knowledge. However, the learning path is still less focused, leading to lower salary benefits when recommending the same number of skills. Compared to the salary-oriented methods, the reward-oriented methods decrease the learning difficulty. However, they still only consider the short-term utility and cannot design a proper and prescient career plan. For example, they replace data mining-related skills with easier skills. However, data mining-related skills could lead to highly paid jobs in the long term.

In contrast, SeSRDQN finds a sustainable and reasonable learning path. Specifically, “Distributed System” and “Framework” tools are fundamental skills that build the anchor of the learning path and can bring instant benefits. “Data Mining” and “Recommender System” help talents to obtain higher abilities in “Data Analysis” on Big Data, where “Machine Learning,” “NLP,” and “Math” provide the theory foundation. It is noticeable that SeSRDQN recommends both theory and practice skills, which shows that our SeSRDQN can effectively guide talents to plan their careers and break the career ceiling.

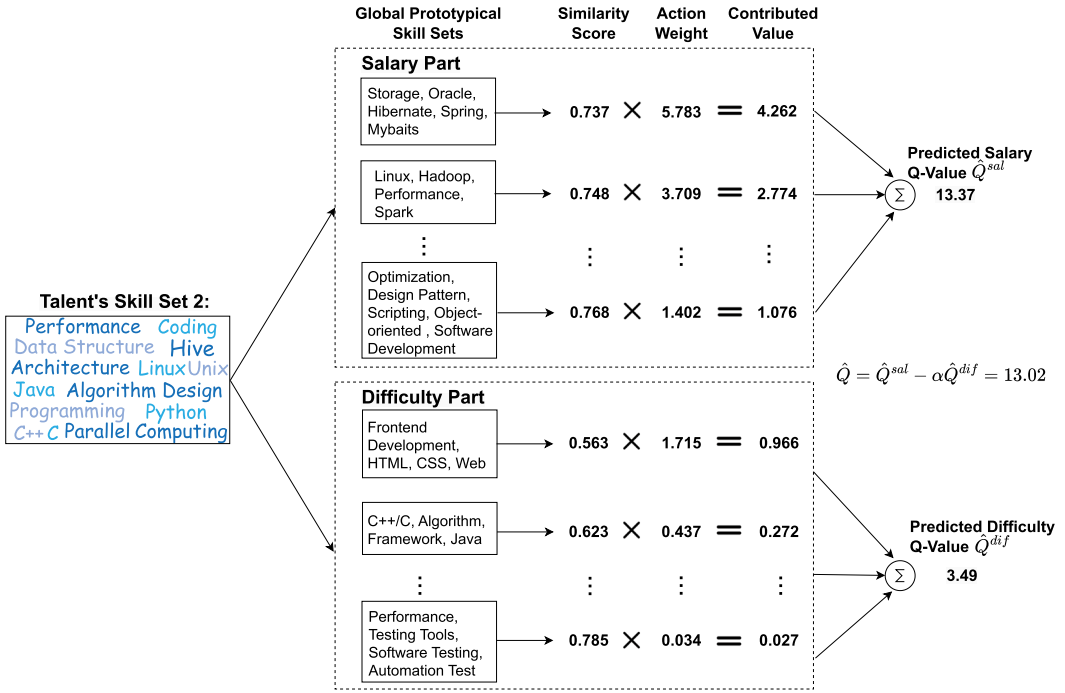


Fig. 14. The reasoning process of SeSRDQN in estimating Q-values.

5.8 RQ6: Case Study on Transparent Reasoning

In Figure 14, we perform the case study to qualitatively evaluate the explainability of the intrinsic reasoning process of SeSRDQN in the second case. For better illustration, we take the recommended skill “Distributed System” with the highest candidate Q-value from the candidate pool for analysis.

There are many observations. Firstly, each prototype reflects a specific skill set pattern. For example, the prototype (“Frontend Development,” “HTML,” “CSS,” and “Web”) is a prevalent skill set pattern for front-end engineers. The combinations of such skill sets provide a global and comprehensive view of the job market from different domains, such as service maintenance, database, software development, and data analytics. Secondly, SeSRDQN disentangles the talent’s skill set by comparing it with the learned prototypes from different perspectives. For example, in the difficulty part, the talent is more likely to the prototypical skill set (“C++/C,” “Algorithm,” “Framework,” and “Java”) since they both concentrate on generic programming skills, in comparison to the frontend development-oriented prototype. In the salary part, the shown similarity scores are relatively close to each other since the talent’s skill set is very generic and has large relevance with database-, performance-, and development-related prototypes. Thirdly, the action weight reflects the learning impact of the candidate skill (“Distributed System”) on the skill set prototypes. In the salary part, storage-related prototype (“Storage,” “Oracle,” “Performance,” and “Spark”) contributes most to the final salary Q-value. It indicates that learning “Distributed System” brings about a remarkable salary increase in terms of storage-related jobs. In the difficulty part, we can find that testing-related prototype (“Performance,” “Testing Tools,” “Software Testing,” and “Automation Test”) contributes little to the final difficulty Q-value, which is behind the intuition that talents with the experience of system development can learn “Distributed System” more easily.

Table 7. User Study Results: 5-Point Likert Scale Ratings of Three Measured Characteristics for Explanatory Methods

Methods	Reward DNN	SRDQN	SeSRDQN-BS	SeSRDQN
Understanding	2.78	3.28	3.42	3.85
Trust	2.95	3.14	3.66	4.19
Usability	2.28	3.57	3.85	4.57

Learning valuable and high-relevance skills can effectively control the learning difficulty and keep the talent career consistent, which can help talents stay competitive in the long term. In the first step, our SeSRDQN reaches the final recommendation by mainly considering the learning utilities of “*Distributed System*” from the *storage* and *data analysis* perspectives, which are critical components of the talent’s current skill set. It explains why our SeSRDQN tends to advise this talent to become an algorithm engineer by learning the skills related to *data mining* and *ML*. By providing the talent with a direct image of the whole reasoning process, the talents can build a comprehensive and global understanding of their skill sets and career development. In this way, SeSRDQN lowers the risk of skill choices and build trust and confidence for the talent.

5.9 RQ6: User Study

Here, we conduct a human-centered user study to evaluate the quality of model explanations and its impact on talent decision-making. Following well-established literature [51], we assessed the user’s comprehension of the model’s reasoning process (Understanding), their confidence and comfort with the recommendations (Trust), and the intuitiveness and ease of use of provided explanations (Usability). We compare our SeSRDQN with Reward DNN, which is a full black-box model without any explanation. Participants are only informed that the model is trained to recommend skills based on a vast amount of online job posting data. We also evaluate SRDQN, which provides the participants with the top-3 candidate skills and their Q-values for reference. SeSRDQN provides explanations by presenting top-influential prototypical skill sets and their relevance to the participant’s current skills. Additionally, we test a SeSRDQN variant (SeSRDQN-BS) using beam search for prototype projection to assess its impact.

To conduct this user study, we recruited 48 postgraduate students and staff members with a background in computer science or electronic engineering from three different universities. Participants were required to have prior knowledge of engineering and have internship/job experience. We asked the participants to complete a list of survey questions after viewing each model’s recommendations with its explanations. The results are shown in Table 7. It can be observed that our SeSRDQN consistently outperforms the other methods across all three perspectives. These findings highlight the necessity of our efforts to enhance Q-value estimation with explanations that are comprehensible to humans. Moreover, traditional DQN methods (SRDQN) provide only minimal insight into the recommendation process, which is insufficient for informed decision-making by talents. SeSRDQN-BS, which employs beam search for prototype projection, shows only modest improvements in understanding and usability compared to SRDQN, suggesting that its more complicated explanations may hinder user experience. In contrast, SeSRDQN achieves substantial improvements in all three aspects, indicating that its explanations offer a more holistic and thorough understanding of the recommendation mechanism. This comprehensive approach aids talents in developing trust and confidence in the outcomes suggested by the model.

Table 8. Performance Evaluation for Pre-training Enhanced Skill Recommendation

Type	Method	4-Step	6-Step	8-Step	10-Step	Salary 12-Step	14-Step	16-Step	18-Step	20-Step	Difficulty
SRDQN	SRDQN	7.03	11.93	17.95	23.57	27.69	31.16	34.07	36.33	38.24	0.66
	- BERT	6.86	11.55	17.40	22.77	27.03	30.43	33.42	35.81	37.86	0.62
	- QWen2	6.83	11.36	17.30	23.17	27.51	30.85	33.67	36.03	37.93	0.62
	- NV-Emb	6.89	11.48	17.39	22.89	26.98	30.58	33.50	35.96	37.98	0.64
	- OpenAI	6.82	11.27	16.98	22.57	27.14	30.87	33.92	36.29	38.12	0.63
SeSRDQN	SeSRDQN	6.56	10.57	16.12	22.10	26.92	31.28	34.12	36.15	38.25	0.65
	- BERT	5.22	8.06	12.47	17.92	23.31	27.87	32.09	35.33	37.83	0.63
	- QWen2	4.77	7.90	12.98	19.22	24.85	29.38	33.00	35.78	37.88	0.65
	- NV-Emb	5.18	9.01	14.07	20.11	25.17	29.64	33.08	35.83	38.27	0.61
	- OpenAI	5.64	9.05	14.18	19.87	25.17	29.38	32.90	35.77	38.04	0.64

The best results are highlighted in *bold and underlined*, and the second-best results are in *bold*.

5.10 RQ7: Evaluation on Pre-trained Model Fusion

In this subsection, we explore the impact of integrating pre-trained models into our approach by incorporating text embeddings from several well-known models, including BERT [22], text-embedding-3-large (OpenAI) [45], gte-Qwen2 [30], and NV-Embed-v2 [24]. These pre-trained embeddings were fused with the learned representations of SRDQN and SeSRDQN through a gated network to assess potential improvements in recommendation performance.

The results are summarized in Table 8. We have two key observations: (1) Despite the additional knowledge and computational cost, pre-trained models did not significantly enhance recommendation performance. This suggests that while these models excel in general language understanding, they may not add substantial value or could even introduce redundancy when combined with the task-specific embeddings already learned by SRDQN and SeSRDQN. The existing skill representations in these models effectively capture relevant patterns, reducing the potential benefit from the inclusion of pre-trained knowledge. (2) Larger models like OpenAI, Qwen2, and NV-Embed-v2 outperformed smaller models like BERT, indicating that smaller models may lack the capacity to generalize effectively in the skill recommendation context. These findings suggest potential future directions, such as exploring job-skill-specific fine-tuning or pre-training methods for improving performance in skill recommendation tasks.

6 Related Works

This article's related work can be summarized into three categories, namely job skill recommendation, deep RL-based applications, and explainable neural networks.

6.1 Job Skill Recommendation

With the development of the knowledge economy, choosing the right skill to learn becomes a crucial problem for the talents' career development [41, 84]. Some works measured market demand to provide talent with skill learning reference [70, 72]. For example, Chootong et al. [12] investigated skill demand from job searching Web sites and recommended in-demand skills to the students. However, due to individual differences, the demanded skills are not always suitable for all talents. In recent years, great efforts have been directed towards recruitment and talent analytics [40, 47, 59, 75]. Alongside advancements in recommendation systems [31, 71, 79], personalized job skill recommendation has garnered significant attention from researchers. Specifically, some works recommend skills according to their relevance to the learner [13, 39, 44, 49, 65]. For example, Dave et al. [13] proposed a method to jointly learn the representation of jobs and skills so that the relevance

of new skills to the learner can be measured. Umemoto et al. [65] designed a progression model to infer the dynamics of skill learning and recommend learning items for upskilling. Meanwhile, some works recommend skills relevant to jobs that the user may be interested in [7, 46]. For example, Pate et al. [46] identified job positions related to the learner's current skill set and recommended additional skills required for these jobs. Different from the above studies, in this article, we explicitly modeled the multi-faceted utilities of the skill-learning process, including the long-term salary and learning difficulty, and provided sustainable skill recommendations based on deep RL.

6.2 Deep RL-based Applications

Deep RL is a technology that combines RL principles with deep learning. It learns to properly act under various states in order to maximize the cumulative reward. In recent years, deep RL has been widely applied in many application scenarios, including search and information retrieval [38, 74, 86], spatial object planning [37, 78], personalized education [84], interior design [77], and recommendation [16, 29, 32, 34, 50, 68, 81]. For different domains, researchers design domain-specific simulated environments and adjust the deep RL models according to the characteristics of the problems. For example, Zhou et al. [86] proposed a generative retrieval model with RL from relevance feedback. In the field of traffic signal control, Zhang et al. [76] designed CityFlow for city-wide traffic simulation by meta-RL. For online recommender systems, user experience or preference is applied as rewards of RL systems [73, 82, 85] and knowledge-aware utility modelling is proposed to improve the recommendation fairness [17] and explainability [29, 67]. For example, Zheng et al. [83] enhanced recommendation systems by capturing fine-grained user feedback, such as user activeness, and developed a deep RL-based model to optimize both immediate and long-term rewards. Different from the previous studies, we design a novel environment to simulate the skill-learning process under different situations and propose an intrinsically explainable job skill recommender. It provides a multi-factorial and exemplar-based explanation for the talent's long-term learning utilities across various skill learning options.

6.3 Explainable Neural Networks

Explainable neural networks target to discover and explain the decision-making process of the neural networks. One research line is to generate explanations for the predictions of black-box models, which is called *post-hoc* methods, including gradient-based methods [3] and perturbation-based methods [35, 36]. The explanation is individually obtained through a pre-existing black-box model. However, some *post-hoc* methods tend to fail to extract intrinsic model insights in many cases [2, 54]. In contrast, another research line is to design self-explainable models [15, 58, 62], which can generate explanations for the predictions during the model training process. Among them, one promising direction is to leverage the concept of prototype learning [28] or case-based reasoning [23]. The main idea is to generate predictions by comparing inputs with learnable prototypes, akin to human cognition process. Many researchers have adopted this idea in different application scenarios for enhancing the model explainability, including image processing [11, 52], sequence classification [42], graph learning [80], and computer games [4, 21, 48].

To summarize, previous works mainly focus on traditional classification tasks and seek to better measure how similar the input is to the prototypes. In contrast, in this article, we focus on explaining path-based skill recommendation and propose a more fine-grained prototype learning method guided by an efficient frequent skill set sampling strategy. In particular, our SeSRDQN focuses on disentangling the complex skill sets with representative frequent subset prototypes and providing intrinsically explainable Q-value estimation for the path-based decision-making process.

7 Conclusion and Discussion

In this article, we introduced a new research problem, namely *sustainable job skill recommendation*, which aims to provide long-sighted, cost-effective, and explainable recommendations for talents who want to learn new job skills. To address this problem, we proposed a data-driven recommendation system based on deep RL. Specifically, we first designed an environment to estimate the skill-learning utilities by mining massive job advertisement data. Then we specially designed a novel SRDQN with a multi-task structure to estimate the long-term skill learning utilities. Furthermore, to enhance recommendation explainability, we constructed a SeSRDQN model. SeSRDQN could reveal the reasoning process of skill-learning utility estimation by providing a weighted combination of similarity scores with representative skill sets. Specifically, we introduced a frequent subset-guided prototyping method, aimed at effectively capturing the prevalent skill sets within the market. To tackle the discrete and combinatorial challenges of the large skill set searching space, we developed a novel MCTS-based optimization-decoding iterative training method. These well-aligned designs made SeSRDQN human-understandable and effectively bridged a key gap in skill recommendation systems, while preserving competitive recommendation performance relative to black-box models. Finally, we conducted extensive experiments on a real-world dataset, which clearly validated the effectiveness and explainability of our approach.

Discussion on the Generalizability of the Method. Our approach, which combines multi-objective deep RL with a self-explaining prototypical network, is well-suited for tasks involving continuous development or strategic acquisition that require balancing long-term utilities. Its built-in explainability fosters user trust by enabling case-based reasoning in complex decision-making contexts. This versatility extends beyond skill recommendation, making it applicable to fields such as financial strategy [25], personalized education (e.g., course recommendations [66]), and urban planning [57], where long-term decision-making and transparency are essential for guiding well-informed and effective decisions.

Future Work. We plan to explore the following directions for extending our proposed approach further. (1) Firstly, we could focus on expanding the multi-objective recommender beyond salary and learning difficulty to address more complex career goals, such as location preferences or specific job types. This would involve integrating additional recruitment data and adapting SeSRDQN into a multi-context system that provides more personalized and flexible recommendations. (2) Secondly, to understand which types of explanations users prefer, it is promising to investigate and compare other explanation methods (e.g., feature importance [54], neural addition [62], versus prototypes) for skill recommendation systems. Additionally, we will explore the impact of different types of explanations on talent skill learning behavior by tracking their preferences, behavior, and feedback. (3) Thirdly, incorporating LLMs could streamline and automate skill set annotation, making the system more scalable and adaptable. Developing job-skill-specific LLMs could possibly further enhance recommendations through their greater capacity and pre-training on diverse datasets.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [2] David Alvarez Melis and Tommi Jaakkola. 2018. Towards robust interpretability with self-explaining neural networks. In *Proceedings of the 32nd Conference on Neural Information Processing Systems*. Curran Associates Inc., 7786–7795.
- [3] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. 2017. Towards better understanding of gradient-based attribution methods for deep neural networks. arXiv:1711.06104. Retrieved from <https://arxiv.org/abs/1711.06104>
- [4] Raghuram Mandyam Annasamy and Katia Sycara. 2019. Towards better interpretability in deep q-networks. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence*, Vol. 33, 4561–4569.

- [5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. arXiv:2309.16609. Retrieved from <https://arxiv.org/abs/2309.16609>
- [6] Timothy T. Baldwin and J. Kevin Ford. 1988. Transfer of training: A review and directions for future research. *Personnel Psychology* 41, 1 (1988), 63–105.
- [7] David Baneres and Jordi Conesa. 2017. A life-long learning recommender system to promote employability. *International Journal of Emerging Technologies in Learning* 12, 06 (2017), 77–93.
- [8] Zahid H. Bhat. 2014. Job matching: The key to performance. *International Journal of Research in Organizational Behavior and Human Resource Management* 2, 4 (2014), 257–269.
- [9] Rowena Blokker, Jos Akkermans, Maria Tims, Paul Jansen, and Svetlana Khapova. 2019. Building a sustainable start: The role of career competencies, career success, and career shocks in young professionals' employability. *Journal of Vocational Behavior* 112 (2019), 172–184.
- [10] Ariel Burstein and Jonathan Vogel. 2017. International trade, technology, and the skill premium. *Journal of Political Economy* 125, 5 (2017), 1356–1412.
- [11] Chaofan Chen, Oscar Li, Daniel Tao, Alina Barnett, Cynthia Rudin, and Jonathan K. Su. 2019. This looks like that: Deep learning for interpretable image recognition. *Advances in Neural Information Processing Systems* 32 (2019), 8928–8939.
- [12] Chalothon Chootong, Timothy K. Shih, Ankhtuya Ochirbat, Worapot Sommoool, W. K. T. M. Gunarathne, and Carl K. Chang. 2019. LCRec: Learning content recommendation (Wiki-based skill book). *Journal of Internet Technology* 20, 6 (2019), 1753–1766.
- [13] Vachik S. Dave, Baichuan Zhang, Mohammad Al Hasan, Khalifeh AlJadda, and Mohammed Korayem. 2018. A combined representation learning approach for better job and skill recommendation. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, 1997–2005.
- [14] Rafael Dix-Carneiro and Brian K. Kovak. 2015. Trade liberalization and the skill premium: A local labor markets approach. *American Economic Review* 105, 5 (2015), 551–57.
- [15] Kevin Fauvel, Fuxing Chen, and Dario Rossi. 2023. A lightweight, efficient and explainable-by-design convolutional neural network for internet traffic classification. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 4013–4023.
- [16] Chao Feng, Defu Lian, Xiting Wang, Zheng Liu, Xing Xie, and Enhong Chen. 2023. Reinforcement routing on proximity graph for efficient recommendation. *ACM Transactions on Information Systems* 41, 1 (2023), 1–27.
- [17] Yingqiang Ge, Shuchang Liu, Ruoyuan Gao, Yikun Xian, Yunqi Li, Xiangyu Zhao, Changhua Pei, Fei Sun, Junfeng Ge, Wenwu Ou, et al. 2021. Towards long-term fairness in recommendation. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, 445–453.
- [18] Marilyn E. Gist, Anna G. Bavetta, and Cynthia Kay Stevens. 1990. Transfer training method: Its influence on skill generalization, skill repetition, and performance level. *Personnel Psychology* 43, 3 (1990), 501–523.
- [19] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, 249–256.
- [20] Dat Hong, Tong Wang, and Stephen Baek. 2023. ProtoryNet-interpretable text classification via prototype trajectories. *Journal of Machine Learning Research* 24, 264 (2023), 1–39.
- [21] Eoin M. Kenny, Mycal Tucker, and Julie Shah. 2022. Towards interpretable deep reinforcement learning with human-friendly prototypes. In *International Conference on Learning Representations*.
- [22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Vol. 1, 2.
- [23] Janet L. Kolodner. 1992. An introduction to case-based reasoning. *Artificial Intelligence Review* 6, 1 (1992), 3–34.
- [24] Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. 2024. NV-Embed: Improved techniques for training LLMs as generalist embedding models. arXiv:2405.17428. Retrieved from <https://arxiv.org/abs/2405.17428>
- [25] Youngbin Lee, Yejin Kim, and Yongjae Lee. 2024. Stock recommendations for individual investors: A temporal graph network approach with diversification-enhancing contrastive learning. arXiv:2404.07223. Retrieved from <https://arxiv.org/abs/2404.07223>
- [26] Haoyuan Li, Yi Wang, Dong Zhang, Ming Zhang, and Edward Y. Chang. 2008. Pfp: Parallel fp-growth for query recommendation. In *Proceedings of the 2008 ACM Conference on Recommender Systems*, 107–114.
- [27] Kuan Li, Yang Liu, Xiang Ao, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2022. Reliable representations make a stronger defender: Unsupervised structure refinement for robust GNN. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 925–935.
- [28] Oscar Li, Hao Liu, Chaofan Chen, and Cynthia Rudin. 2018. Deep learning for case-based reasoning through prototypes: A neural network that explains its predictions. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence* 32 (2018), 3530–3537.

- [29] Yicong Li, Hongxu Chen, Yile Li, Lin Li, PhilipS Yu, and Guandong Xu. 2023. Reinforcement learning based path exploration for sequential explainable recommendation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, 11 (2023), 11801–11814. DOI: <https://doi.org/10.1109/TKDE.2023.3237741>
- [30] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards general text embeddings with multi-stage contrastive learning. arXiv:2308.03281. Retrieved from <https://arxiv.org/abs/2308.03281>
- [31] Yongquan Liang, Qiuyu Song, Zhongying Zhao, Hui Zhou, and Maoguo Gong. 2023. BA-GNN: Behavior-aware graph neural network for session-based recommendation. *Frontiers of Computer Science* 17, 6 (2023), 176613.
- [32] Hao Liu, Qian Gao, Jiang Li, Xiaochao Liao, Hao Xiong, Guangxing Chen, Wenlin Wang, Guobao Yang, Zhiwei Zha, Daxiang Dong, et al. 2021. Jizhi: A fast and cost-effective model-as-a-service system for web-scale online inference at Baidu. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 3289–3298.
- [33] Yang Liu, Xiang Ao, Fuli Feng, and Qing He. 2022. UD-GNN: Uncertainty-aware debiased training on semi-homophilous graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1131–1140.
- [34] Zirui Liu, Jiejie Tian, Qingpeng Cai, Xiangyu Zhao, Jingtong Gao, Shuchang Liu, Dayou Chen, Tonghao He, Dong Zheng, Peng Jiang, et al. 2023. Multi-task recommendations with reinforcement learning. In *Proceedings of the ACM Web Conference 2023*, 1273–1282.
- [35] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. 2020. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence* 2, 1 (2020), 56–67.
- [36] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* 30 (2017), 4765–4774.
- [37] Hui Luo, Jingbo Zhou, Zhifeng Bao, Shuangli Li, J. Shane Culpepper, Haochao Ying, Hao Liu, and Hui Xiong. 2020. Spatial object recommendation with hints: When spatial granularity matters. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 781–790.
- [38] Ziyao Luo, Linfeng Zhao, Wei Cheng, Sihao Chen, Qi Chen, Hui Xue, Haidong Wang, Chuanjie Liu, Mao Yang, and Lintao Zhang. 2021. Match plan generation in web search with parameterized action reinforcement learning. In *Proceedings of the Web Conference 2021*, 1040–1052.
- [39] Abhinav Maurya and Rahul Telang. 2017. Bayesian multi-view models for member-job matching and personalized skill recommendations. In *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data)*. IEEE, 1193–1202.
- [40] Qingxin Meng, Keli Xiao, Dazhong Shen, Hengshu Zhu, and Hui Xiong. 2022. Fine-grained job salary benchmarking with a nonparametric Dirichlet process-based latent factor model. *INFORMS Journal on Computing* 34, 5 (2022), 2443–2463.
- [41] Qingxin Meng, Hengshu Zhu, Keli Xiao, Le Zhang, and Hui Xiong. 2019. A hierarchical career-path-aware neural network for job mobility prediction. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 14–24.
- [42] Yao Ming, Panpan Xu, Huamin Qu, and Liu Ren. 2019. Interpretable and steerable sequence learning via prototypes. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 903–913.
- [43] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fiedelnd, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.
- [44] Xiang Qian Ong and Kwan Hui Lim. 2023. SkillRec: A data-driven approach to job skill recommendation for career insights. In *Proceedings of the 2023 15th International Conference on Computer and Automation Engineering (ICCAE)*. IEEE, 40–44.
- [45] OpenAI. 2023. Embeddings Guide - OpenAI Platform. Retrieved September 14, 2024 from <https://platform.openai.com/docs/guides/embeddings>
- [46] Bharat Patel, Varun Kakuste, and Magdalini Eirinaki. 2017. CaPaR: A career path recommendation framework. In *Proceedings of the 2017 IEEE 3rd International Conference on Big Data Computing Service and Applications*, 23–30.
- [47] Chuan Qin, Hengshu Zhu, Dazhong Shen, Ying Sun, Kaichun Yao, Peng Wang, and Hui Xiong. 2023. Automatic skill-oriented question generation and recommendation for intelligent job interviews. *ACM Transactions on Information Systems* 42, 1 (2023), 1–32.
- [48] Ronilo Ragodos, Tong Wang, Qihang Lin, and Xun Zhou. 2022. ProtoX: Explaining a reinforcement learning agent via prototyping. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 35, 27239–27252.
- [49] Siddharth Reddy, Igor Labutov, and Thorsten Joachims. 2016. Learning student and content embeddings for personalized lesson sequence recommendation. In *Proceedings of the 3rd (2016) ACM Conference on Learning @ Scale*. ACM, New York, NY, 93–96.
- [50] Zhaochun Ren, Na Huang, Yidan Wang, Pengjie Ren, Jun Ma, Jiahuan Lei, Xinlei Shi, Hengliang Luo, Joemon Jose, and Xin Xin. 2023. Contrastive state augmentations for reinforcement learning-based recommender systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 922–931.

- [51] Yao Rong, Tobias Leemann, Thai-Trang Nguyen, Lisa Fiedler, Peizhu Qian, Vaibhav Unhelkar, Tina Seidel, Gjergji Kasneci, and Enkelejda Kasneci. 2024. Towards human-centered explainable AI: A survey of user studies for model explanations. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 4 (April 2024), 2104–2122. DOI: <https://doi.org/10.1109/tpami.2023.3331846>
- [52] Dawid Rymarczyk, Łukasz Struski, Michał Górszczak, Koryna Lewandowska, Jacek Tabor, and Bartosz Zieliński. 2022. Interpretable image classification with differentiable prototypes assignment. In *Proceedings of the European Conference on Computer Vision*. Springer, 351–368.
- [53] Dawid Rymarczyk, Łukasz Struski, Jacek Tabor, and Bartosz Zieliński. 2021. Protopshare: Prototypical parts sharing for similarity discovery in interpretable image classification. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 1420–1430.
- [54] Maresa Schröder, Alireza Zamanian, and Narges Ahmadi. 2023. Post-hoc saliency methods fail to capture latent feature importance in time series data. In *Proceedings of the International Workshop on Trustworthy Machine Learning for Healthcare*. Springer, 106–121.
- [55] Dazhong Shen, Hengshu Zhu, Keli Xiao, Xi Zhang, and Hui Xiong. 2023. Exploiting connections among personality, job position, and work behavior: Evidence from joint Bayesian learning. *ACM Transactions on Management Information Systems* 14, 3 (2023), 1–20.
- [56] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. 2017. Mastering the game of go without human knowledge. *Nature* 550, 7676 (2017), 354–359.
- [57] Yunita Sopiana and Muhammad Ade Kurnia Harahap. 2023. Sustainable urban planning: A holistic approach to balancing environmental conservation, economic development, and social well-being. *West Science Interdisciplinary Studies* 1, 02 (2023), 43–53.
- [58] Ying Sun, Hengshu Zhu, Chuan Qin, Fuzhen Zhuang, Qing He, and Hui Xiong. 2021. Discerning decision-making process of deep neural networks with hierarchical voting transformation. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 34, 17221–17234.
- [59] Ying Sun, Hengshu Zhu, Lu Wang, Le Zhang, and Hui Xiong. 2024. Large-scale online job search behaviors reveal labor market shifts amid COVID-19. *Nature Cities* 1, 2 (2024), 150–163.
- [60] Ying Sun, Fuzhen Zhuang, Hengshu Zhu, Qing He, and Hui Xiong. 2021. Cost-effective and interpretable job skill recommendation with deep reinforcement learning. In *Proceedings of the Web Conference 2021*, 3827–3838.
- [61] Ying Sun, Fuzhen Zhuang, Hengshu Zhu, Qi Zhang, Qing He, and Hui Xiong. 2021. Market-oriented job skill valuation with cooperative composition neural network. *Nature Communications* 12, 1 (2021), 1992.
- [62] Omer Tal, Yang Liu, Jimmy Huang, Xiaohui Yu, and Bushra Aljbawi. 2019. Neural attention frameworks for explainable recommendation. *IEEE Transactions on Knowledge and Data Engineering* 33, 5 (2019), 2137–2150.
- [63] Surendra Babu Talluri, Nishant Uppal, Jos Akkermans, and Alexander Newman. 2024. Towards a self-regulation model of career competencies: A systematic review and future research agenda. *Journal of Vocational Behavior* 149 (March 2024), 103969. DOI: <https://doi.org/10.1016/j.jvb.2024.103969>
- [64] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from <https://arxiv.org/abs/2302.13971>
- [65] Kazutoshi Umemoto, Tova Milo, and Masaru Kitsuregawa. 2020. Toward recommendation for upskilling: Modeling skill improvement and item difficulty in action sequences. In *Proceedings of the IEEE International Conference on Data Engineering*, 169–180.
- [66] Chao Wang, Hengshu Zhu, Peng Wang, Chen Zhu, Xi Zhang, Enhong Chen, and Hui Xiong. 2021. Personalized and explainable employee training course recommendations: A Bayesian variational approach. *ACM Transactions on Information Systems* 40, 4 (2021), 1–32.
- [67] Xiting Wang, Yiru Chen, Jie Yang, Le Wu, Zhengtao Wu, and Xing Xie. 2018. A reinforcement learning framework for explainable recommendation. In *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)*. IEEE, 587–596.
- [68] Xiting Wang, Kunpeng Liu, Dongjie Wang, Le Wu, Yanjie Fu, and Xing Xie. 2022. Multi-level recommendation reasoning over knowledge graphs with reinforcement learning. In *Proceedings of the ACM Web Conference 2022*, 2098–2108.
- [69] Christopher John Cornish Hellaby Watkins. 1989. *Learning from Delayed Rewards*. University of Cambridge.
- [70] Xunxian Wu, Tong Xu, Hengshu Zhu, Le Zhang, Enhong Chen, and Hui Xiong. 2019. Trend-aware tensor factorization for job skill demand analysis. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 3891–3897.
- [71] Haoran Xin, Xinjiang Lu, Nengjun Zhu, Tong Xu, Dejing Dou, and Hui Xiong. 2022. Captor: A crowd-aware pre-travel recommender system for out-of-town users. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1174–1184.

- [72] Tong Xu, Hengshu Zhu, Chen Zhu, Pan Li, and Hui Xiong. 2018. Measuring the popularity of job skills in recruitment market: A multi-criteria approach. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2572–2579.
- [73] Surong Yan, Chenglong Shi, Haosen Wang, Lei Chen, Ling Jiang, Ruilin Guo, and Kwei-Jay Lin. 2024. Teach and explore: A multiplex information-guided effective and efficient reinforcement learning for sequential recommendation. *ACM Transactions on Information Systems* 42, 5 (2024), 1–26.
- [74] Jing Yao, Zhicheng Dou, Jun Xu, and Ji-Rong Wen. 2020. RLPer: A reinforcement learning model for personalized search. In *Proceedings of the Web Conference 2020*, 2298–2308.
- [75] Yuyang Ye, Hengshu Zhu, Tianyi Cui, Runlong Yu, Le Zhang, and Hui Xiong. 2024. University evaluation through graduate employment prediction: An influence based graph autoencoder approach. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 7255–7267. DOI : <https://doi.org/10.1109/TKDE.2024.34022>
- [76] Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. 2019. CityFlow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *Proceedings of the World Wide Web Conference*, 3620–3624.
- [77] He Zhang, Ying Sun, Weiyu Guo, Yafei Liu, Haonan Lu, Xiaodong Lin, and Hui Xiong. 2023. Interactive interior design recommendation via coarse-to-fine multimodal reinforcement learning. In *Proceedings of the 31st ACM International Conference on Multimedia*, 6472–6480.
- [78] Weijia Zhang, Hao Liu, Hui Xiong, Tong Xu, Fan Wang, Haoran Xin, and Hua Wu. 2022. RLCharge: Imitative multi-agent spatiotemporal reinforcement learning for electric vehicle charging station recommendation. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2022), 6290–6304.
- [79] Zhiqian Zhang, Chenliang Li, Zhiyong Wu, Aixun Sun, Dengpan Ye, and Xiangyang Luo. 2020. Next: A neural network framework for next POI recommendation. *Frontiers of Computer Science* 14 (2020), 314–333.
- [80] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Cheekong Lee. 2022. ProtGNN: Towards self-explaining graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36, 9127–9135.
- [81] Kangzhi Zhao, Xiting Wang, Yuren Zhang, Li Zhao, Zheng Liu, Chunxiao Xing, and Xing Xie. 2020. Leveraging demonstrations for reinforcement recommendation reasoning over knowledge graphs. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 239–248.
- [82] Xiangyu Zhao, Changsheng Gu, Haoshenglun Zhang, Xiwang Yang, Xiaobing Liu, Jiliang Tang, and Hui Liu. 2021. Dear: Deep reinforcement learning for online advertising impression in recommender systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, 750–758.
- [83] Guanjie Zheng, Fuzheng Zhang, Zihan Zheng, Yang Xiang, Nicholas Jing Yuan, Xing Xie, and Zhenhui Li. 2018. DRN: A deep reinforcement learning framework for news recommendation. In *Proceedings of the World Wide Web Conference*, 167–176.
- [84] Zhi Zheng, Ying Sun, Xin Song, Hengshu Zhu, and Hui Xiong. 2023. Generative learning plan recommendation for employees: A performance-aware reinforcement learning approach. In *Proceedings of the 17th ACM Conference on Recommender Systems*, 443–454.
- [85] Sijin Zhou, Xinyi Dai, Haokun Chen, Weinan Zhang, Kan Ren, Ruiming Tang, Xiuqiang He, and Yong Yu. 2020. Interactive recommender system via knowledge graph-enhanced reinforcement learning. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development In Information Retrieval*, 179–188.
- [86] Yujia Zhou, Zhicheng Dou, and Ji-Rong Wen. 2023. Enhancing generative retrieval with reinforcement learning from relevance feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 12481–12490.

Received 5 January 2024; revised 22 September 2024; accepted 12 November 2024